

RESEARCH ARTICLE

Multi-Modal Comparative Analysis on Execution of Phishing Detection Using Artificial Intelligence

DIVYA JENNIFER DSOUZA¹, ANISHA P. RODRIGUES¹, AND ROSHAN FERNANDES²¹Department of Computer Science and Engineering, NMAM Institute of Technology, NITTE (Deemed to be University), Nitte 574110, India²Department of Cyber Security, NMAM Institute of Technology, NITTE (Deemed to be University), Nitte 574110, India

Corresponding author: Anisha P. Rodrigues (anishapr@nitte.edu.in)

This work was supported and funded by NITTE (Deemed to be University), India.

ABSTRACT Phishing is the process of deceiving or stealing private or confidential information through illicit means. This could lead to financial loss, loss of reputation, and identity theft. Hence, identifying and preventing the use of such phishing sites becomes crucial. In data science, the term outlier, also termed an anomaly refers to points or series of data that opt out of the normal behaviour of the system under study. Anomaly detection touches down on the concepts related to studying the authentic outlier in a data set. The paper aims to present the optimised techniques and multiple modes of executing the process for detecting phishing websites. The most relevant features are chosen for execution by applying feature extraction. The Mendley phishing websites dataset is used to detect phishing websites, along with the SPAM-HAM publicly available dataset, which is used for detecting SPAM/HAM classification for SMS data in this research study. The experiments are also carried out on a custom dataset to avoid any bias present in a publicly available dataset. The study is carried out in three modes, namely offline, batch, and incremental, using machine learning models. The performance evaluation metrics such as accuracy, f1 score, precision, recall, and time complexity of the machine learning models and accuracy and loss metrics of the deep learning models are compared between the different modes. The study is then summarised by detailing the pros and cons of each of the modes and models used for the study. The incremental mode of execution suits better for real-time processing, with an accuracy of 97.1% on the custom dataset using the adaptive random forest (ARF) classifier available in the Python's River Framework. But if we make use of the deep learning approach with Keras sequential model, the accuracy obtained was 99.28%.

INDEX TERMS Outlier detection, anomaly, phishing, incremental learning, feature extraction, batch processing, spam, machine learning.

I. INTRODUCTION

Phishing is the technique used by attackers to dupe netizens, common men, and business people. Phishing is a sort of attack in the social engineering sector aimed at individuals and business organisations by sending fraudulent communications that look legitimate by various means like emails, phone calls, SMS, illicit websites, links, pop-up URLs, and many more, with the sole intention to steal or jeopardise user data or reputation [1]. The main channels of phishing are emails, wherein attackers posing as genuine senders trick users into entering their confidential details, namely

passwords, mpins, and OTPs. Phishing can also be done through smart phones, called smishing. Attackers send fake SMS, URLs, and links, which, if clicked, can cause users to lose their valuable information to the hackers. The URLs of phishing websites may be very similar to those of genuine websites to the human eye, but they differ in their IP [2]. There are various types of phishing attacks, namely spear phishing, smishing, vishing, email phishing, whaling, and angler phishing. It is also very evident that many users face a challenge in day-to-day life regarding the increase in numbers of spam SMS messages [3], emails, ads, and many more. Hence, identifying and curbing these attacks becomes important to safeguard user data. With the advent of the internet, modern technologies, and tools, the rate at which

The associate editor coordinating the review of this manuscript and approving it for publication was Massimo Cafaro¹.

data is generated is exponential. Also, cases where users put their crucial data into the digital world without proper safety measures have led to an increase in fraud. Currently, most of the data is real-time or streaming. In other words, one can call this time series data. The mining of time series data deals with extracting useful data streams and meaningful patterns to analyse in real time. This becomes a very challenging task, especially when it involves evolving streams of data or big data constraints like storage and quick processing. Making decisions depends on how fast it can detect irregularities and malfunctions, especially in streaming implementations like network intrusions, fraudulent apps, failures from critical maintenance tools, and medicinal fields like disease diagnosis. Hence, detecting these anomalies becomes a vital component for businesses whose processing depends on continuous streaming from real-time data events. Outlier detection for streaming data refers to possible methods of efficiently handling the real-time flow of continuous data and, at the same time, extracting value-based knowledge and information in decision-making to segregate good and anomalous content by assigning appropriate anomaly scores. In the literature, a lot of frameworks have been proposed in the area of real-time data processing and anomaly detection. Popular ones are Sci-kit-Multi-Flow, which is an open source ML framework for data streaming, multi-labelling, and multi-output. It is implemented using the Python language and includes various algorithms and techniques for data-stream processing for streaming data. Apart from these frameworks, there is also Creme Library, PySAD, and PySpark from the Python community. From the big data end, there are Kafka, Flume, and Apache Spark for dealing with streaming data streams. River is a Python package used for processing streaming data. In River, the features of a sample are stored inside a dictionary, which in Python is called a dict and is a native data structure. The main advantage of using plain dicts is that it removes the overhead that comes with data structures. This is important in a streaming context because it would be possible to process many individual samples in rapid succession. Another advantage is that, since it is not typed, it can store heterogeneous data.

A. MOTIVATION FOR THE WORK

Looking at the literature for big data streaming frameworks, River, Creme, and Scikit-Multiflows are the latest and most promising ones in the area since they are used to implement almost all stream learning techniques using the popular Python language, which has different learning algorithms and streaming techniques with good solutions to address time and space constraints. This research aims to carry out research on the topic of detection of phishing websites and SPAM/HAM SMS classification in business practices, with a particular focus on their strategies and comparing their performance in various modes of execution. As prevalent, there is a rise in online theft and cybercrime. The motivation for choosing to carry out this research project is to find innovative solutions to curb fraudulent activities and protect user identities and other

private information while transacting online. This project focuses on reducing the time and space complexities by applying efficient algorithms to optimise the process after preprocessing the relevant attributes. The main objectives of this research include:

1. Comparative analysis of the various modes of execution, which are offline, batch, and incremental processing, using a publicly available phishing dataset and SMS spam ham dataset to get an insight on how phishing website detection performs in various modes of execution to evaluate performance metrics.
2. Analysis to show that removing irrelevant features or adding optimised attributes can greatly impact the time complexity and accuracy of the model.
3. Varying batch sizes in batch mode execution to analyse their impact on the overall performance of the data. The novelty of the research include:

1. Use of a custom dataset for phishing data to avoid any potential bias. The procedure to collect custom data and extract its features is explained in Section III-A3.
2. Prioritising critical features by applying ensemble tuned feature extraction methods (explained in the Data Preprocessing Section of Custom Dataset III-A3) to check their impact on performance evaluation.

This paper is organised as follows:

Section I introduces the topic under study, the work accomplished so far, and the motivations.

Section II provides a review of the works carried out for the detection of phishing data sets and spam/ham SMS data sets, as well as works related to real-time processing and batch processing. A detailed survey and classification of anomaly detection techniques applied to streaming data are listed. Recent works in the field of phishing detection are also summarized.

Section III gives a description of methodology, dataset and parameters, experimental setup and modes of processing.

Section IV gives experimental evaluation results and performance metrics on the phishing URL dataset, the spam/ham dataset for SMS, and the custom data.

Section V gives a conclusion to our work by opening future scopes.

II. RELATED WORKS

Firstly, we chose to carry out our own primary research to complement pre-existing papers on this topic. The survey paper by Tang and Mahmoud [4] explains the various ways to obtain legitimate and phishing websites, the URL dissection to identify suspicious URLs, the various techniques used for detection, their accuracy, time and space complexities, and the pros and cons of each method. Rao and Pais [7] introduced Jail-Phish, an advanced search engine to detect phishing that gave a TNR of 99.36 percent on an unpopular legitimate set and a TPR of 97.77 percent on an old and new phishing set. Alialjofey et al. [8] introduced an effective detection technique for phishing websites that uses URL and HTML attributes and achieved an accuracy of

TABLE 1. Related works to phishing website detection.

Author & Year	Dataset	Technique	Keypoints/Limitations
Furqan et. al [25] 2023	'Spam or Ham' EMP 'Spam filter' Kaggle	TF-IDF BOW	Combine multiple features than single features Small size of imbalanced dataset
Melgar et. al [26] 2023	Public Dataset Spanish electricity demand data	Nearest-Neighbors forecasting algorithm	Incremental learning approach Not tested on multivariable time series datasets
Li A et. al [27] 2024	Image datasets: OrganA CIFAR100-C, Omniglot	Adaptive Centered Representations (ACR)	Zero-shot batch-level anomaly detection Method depends on assumptions
He X et. al [28] 2023	Synthetic/Real-world STD, TSAD, TSF	OneShotSTL	Decompose time series online Cannot handle missing points, irregular samples
Orunsolu et. al [29] 2021	Suspicious URLs	Invocation Connector Atomic Unit	Incremental Transaction Identification Module Tested with Few Features
Giri S et. al [30] 2023	Tiago's Dataset	NLP, CNN, BUNOW GloVe word embedding	Multiple deep neural network models GloVe doesnot achieve best Accuracy
Prasad A et. al [31] 2024	PhishTank, OpenPhish, MalwareWorld)	Similarity Index Incremental learning	PhiUSIIL Risk of over-training
Jamal S et. al [32] 2023	Two opensource email, SPAM -HAM data	DistilBERT RoBERTA	Large Language Model Approach Requires additional tuning
Pandey et. al [49] 2023	OpenPhish Alexa	RF, LR, DT SVM, NB	Uses Websites Color Palette Depends on Screenshot clarity
Ariyadasa et. al [50] 2024	Mendeley	Reinforcement Deep Learning	Knowledge acquisition process(KAP) Vulnerable to Dynamic attacks
Bountakas et. al [51] 2023	Custom Dataset	Stacking Soft voting	Innovative evaluation guidelines Imbalanced Datasets

96.76 percent with only 1.39 percent false positive rate on their own dataset and an accuracy of 98.48 percent with 2.09 percent false positive rate on a benchmark dataset.

In [4], the authors have elaborately surveyed the various links, techniques, and models for phishing website detection and also presented their performances and analyses. In [6], the

TABLE 2. Related works to Phishing Website Detection (contd).

Author & Year	Dataset	Technique	Keypoints/Limitations
Mourtaji et. al [52] 2021	Custom Dataset	CART, SVM, CNN	Logical RuleBase UnReliable RealTime
van Geest et. al [45] 2024	Public dataset PILWD-134 K	Hybrid Framework	Makes use of multiple models Individual models don't yield same results
Kritika et. al [46] 2024	Public and Custom Datasets	Deep Learning	Review of research works from 2019 to 2024
Erzhou et. al [47] 2024	PhishTank, OpenPhish, Alexa, Common Crawl	Hybrid Approach PDHF	Automatic Deep Features
Dam Minh et. al [48] 2024	Malicious URL Dataset	Deep Learning	Deep Learning techniques by Extensions in web browser

authors have proposed a multidimensional feature-oriented phishing detection technique based on a fast detection method using a deep learning approach. Initially, the sequence of character features of the given URL is used for quick classification by extracting features using a deep learning model that does not require domain knowledge or any third-party assistance. In the next phase, the output of the first phase, along with URL statistics, web code features, and web page text features, is mapped into multidimensional features. In [9], authors have proposed a convolutional neural network architecture using a fast AI library across GPUs to overcome the shortcomings of traditional neural networks. In [10], authors have introduced a deep learning framework called PhishGNN, using a graph neural network that uses the hyperlink graph structure of websites. The accuracy claimed was 99.7%. According to the authors in [11], social web engineering attacks and phishing could be reduced by four metal learner models, namely Bagging Extra Tree (BET), AdaBoost Extra Tree (ABET), Rotation Forest Extra Tree (RoFBET), and Logit Boost Extra Tree (LBET), developed using LogitBoost Extra Tree. The models achieved an accuracy of 97% with a drastically low false positive rate of 0.028.

Tables 1 and 2 gives a summary of the papers referred to, along with the datasets used, methodologies used, key features of the study, and its limitations.

A. RECENT WORKS

The authors in [45] focus on hybrid approaches to solving phishing attacks. Their approach can bypass the limitations of

single usage models such as language, training time, models working as black boxes, and robustness by considering three features: URL, HTML, and DomTree, which give an accuracy of 97.44%. In [46], researchers have given a comprehensive review of deep learning approaches to detecting phishing URLs. The reviews carried out are based on feature engineering techniques like word embedding, character-level representations, and features that are handcrafted. The article [47] highlights effective feature selection methods and also proposes a new technique called PDHF, which stands for phishing detection hybrid features, to automate generating deep features. PDHF claims to have an accuracy 99.65%, precision of 99.42%, recall 99.40%, and F1-score of 99.41%. The authors in [48] give their findings in real-time phishing detection using deep learning methods by tuning the browser extensions that protect users from untrusted connections. The experiment was conducted on a dataset containing malicious URLs, with 651,191 samples of URLs. The proposed web browser extension technique gave an accuracy of 98.4% with Convolution Neural Network. Most of the models discussed so far require the user to have a minimum domain knowledge of the dataset under consideration. In [12], the authors have proposed an automatic model that captures the semantic or sequential patterns of the URLs. The authors have compared various ML models and concluded that SVM works better for their dataset. In [13], authors have come up with a novel approach to detecting phished URLs using character-level word encoding using deep learning. The experimental results showed true positive instances of 98.12%. It was also claimed that the proposed method operates well even

in energy-saving modes of phishing detection systems. The authors in [14] have introduced a novel technique called PhiBoost, which uses an adaptive boosting approach. The feature selection methods used were correlation-based and BFS greedy hill climbing. The accuracy obtained was 98.7%. The authors in [15] have performed a systematic survey of literature reviews on the latest trends and techniques that are used to curb phishing activities. The authors in [16] have proposed an identity-based hybrid technique that is based on text identity with visual web pages. The experiment was tested on multiple datasets. The overall performance accuracy obtained was 98.6%. In [17], authors give importance to deep learning techniques to address the issue of fraud done through phishing. The authors combined the CNN and LSTM algorithms and named the hybrid the Intelligent Phishing Detection System (IPDS). It was then applied to larger datasets, and the accuracy obtained through this model was 93.28% with an average time detection of 25 seconds. In [18], the authors reviewed the cloaking techniques used for phishing to evade detection by anti-phishing entities. The authors in [19] have introduced a hybrid system using supervised and unsupervised machine learning techniques to curb SMS spam, which tricks users into giving away confidential information, which might lead to severe consequences. Other interesting studies include the BiLSTM approach to detecting spam SMS, the TFIDF and voting classifier [20] approach, word embedding techniques N grammes, Bag of Words (BOW), Word2Vec, and Doc2Vec citeb21. In papers [56], [57], [58], and [59] researchers discuss the importance of quantization techniques applied on deep learning models to extend the application on resource-constrained scenarios to boost computational efficiency or inference speed.

B. LITERATURE GAP

As seen in the related works, one can conclude that even though efforts are being made to detect phishing websites, there is still a lot of scope for improvement. The focus is on the following aspects to improve performance:

1. High-Quality Data Set: Having strong domain knowledge about the data is essential to preprocess the data. Hence, a custom dataset is used to compare the study with the existing dataset to eliminate any kind of potential bias that might be prevalent in a publicly available dataset.

2. Efficient feature extraction, selection, and fine tuning parameters: processing irrelevant features or including only necessary or optimal features affects the performance of the outcome. Also, the models applied for classification work better if we can fine-tune the parameters relevant to the data set.

3. Response Time for Real-Time Systems: Any classifying model is effective only if it can classify or detect appropriately within the stipulated time as per the needs of the user. The various modes used for execution, such as offline, batch, and incremental, are summarised to indicate the optimised modes.

Tables 19 and 20 give summarised details of the state-of-the-art comparison of the datasets used by various authors, including the number of features used, the feature selection methods, classifier models, and accuracy and performance measures.

III. METHODOLOGY

The following section describes the datasets and parameters used in the study, the experimental setup, modes of processing, techniques used, and the architectural flow of the process.

A. DATA

1) PHISHING DATASET

The phishing dataset used for the study is taken from a publicly available domain called the Mendley dataset [44] and is available for reproducing the experiment at [53]. The dataset consists of 11055 total samples of website URLs with 32 features. The features extracted are from different aspects of the URL (Uniform Resource Locator) by verifying if the URL adheres to the standards that need to be followed set by Google called RFC 3986, which is a generic syntax that identifies an abstract or physical resource. RFC 3986 also sets rules and considerations for security for the use of URIs on the internet.

Data Preprocessing: The preprocessing of data is a crucial process in order to get accurate results. For the Mendley phishing dataset, the following are the steps carried out:

- i) Verified if data was balanced (it was), normalized using standard scalar, and eliminated the possibility of finding null or missing values since the data did not have any null values.
- ii) The feature extraction used was heat map correlation to rule out irrelevant features. The term irrelevant is used to rule out the possibility of using features that do not contribute to enhancing the accuracy of the model. A heat map aids in selecting priority features by giving a correlation between all available features and the target variable. It also aids in showing the direction and strength of the correlation between each pair of features, which assists the researcher in making a decision about choosing the attributes. For the Mendley dataset, there were nine negatively correlated features, which were eliminated. Hence, the reduced set with twenty-three features was considered for the process.

2) SMS SPAM/HAM DATASET

The second type of data is the SMS dataset used for the study, taken from the publicly available UCI Machine Learning Repository [24]. The SMS Spam Collection is a public set of SMS-labelled texts that are collected for mobile phone spam research. It has SMS messages in English of around 5,574 messages, tagged as either ham (legitimate) or spam. The document contains a single message per line, and every line is made of two columns: c1 contains the label (ham or spam), and c2 contains the raw text. The dataset is available for reproducing at [55].

Data Preprocessing: The following preprocessing steps are followed for the SMS dataset: Correlation: For the

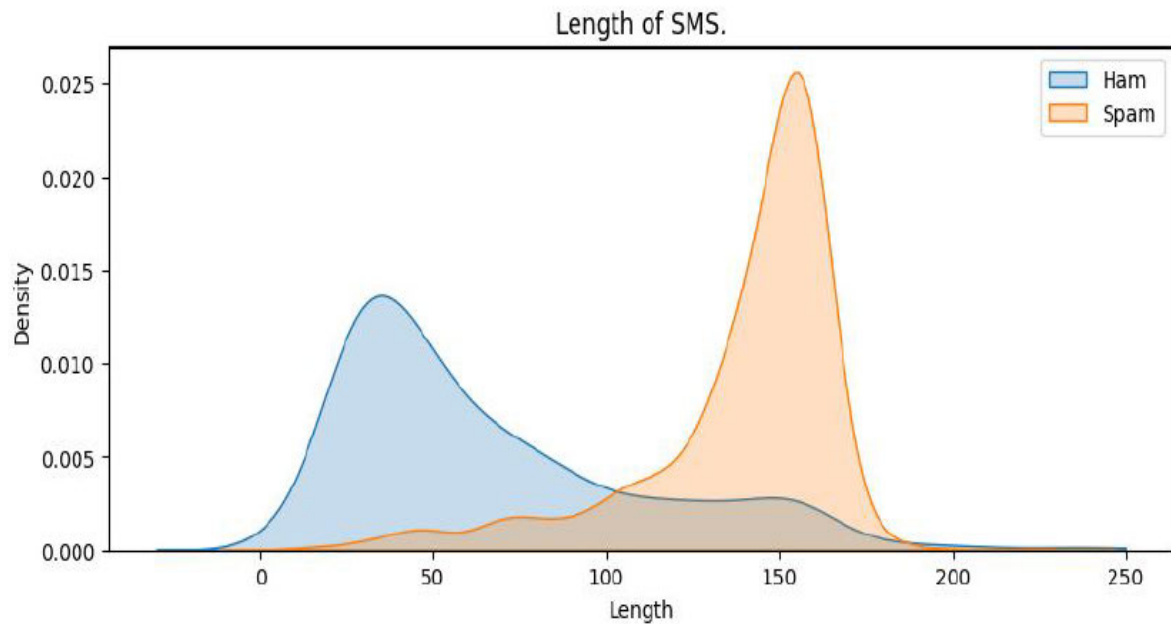


FIGURE 1. Feature extraction from URL.

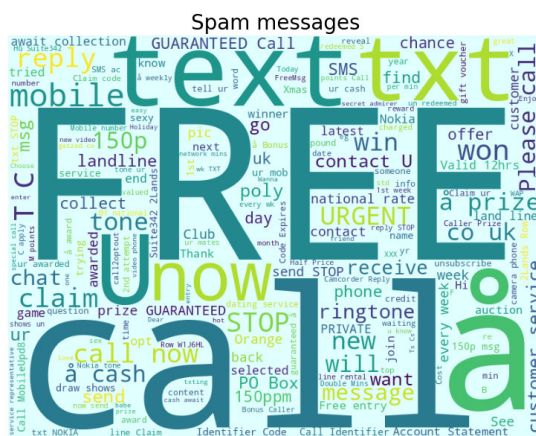


FIGURE 2. Word cloud of spam messages.



FIGURE 3. Word cloud of Ham messages.

SPAM/HAM detection, we only have one column as a feature for the text and one target column for classification. Hence, there is no need for correlation.

SMS Length: The Fig.1 shows the length of SMS. This process is done to see how spam and ham messages differ, usually in their length. It can be observed that spam messages are longer than ham messages in general (due to the number of words). Spam messages have around 150 characters in the dataset considered for the research.

Word-Cloud: Also termed as tag clouds, are graphical representations of word frequencies that give greater importance to words that appear more frequently. Figs.2 and 3 show the word cloud for spam and ham messages. **Punctuation and Stopwords:** The preprocessing steps for common NLP tasks are to convert text to lowercase and to remove

punctuation and stop-words. Punctuation like “#\$%&'()*+,-./:;<=>?@[\\]^_‘{|}” can be ignored.

The words that are commonly used, like “and,” “the,” and “him,” which are assumed to be equivocal in representing the content of a text, could be removed. This can prevent them from being used as signals for prediction, which are called stop-words. Further steps, especially for text classification tasks, are tokenization, vectorization, and TF-IDF weighting.

3) CUSTOM DATASET

To eliminate any kind of potential bias that could be prevalent in the publicly made available dataset, a custom dataset was used for the study and was prepared by the researchers. Fig. 4 shows the flow diagram for how features are extracted from the URLs collected from various sites like Alexa, OpenPhish, PhishTank, and so on. The generated rules and the way the

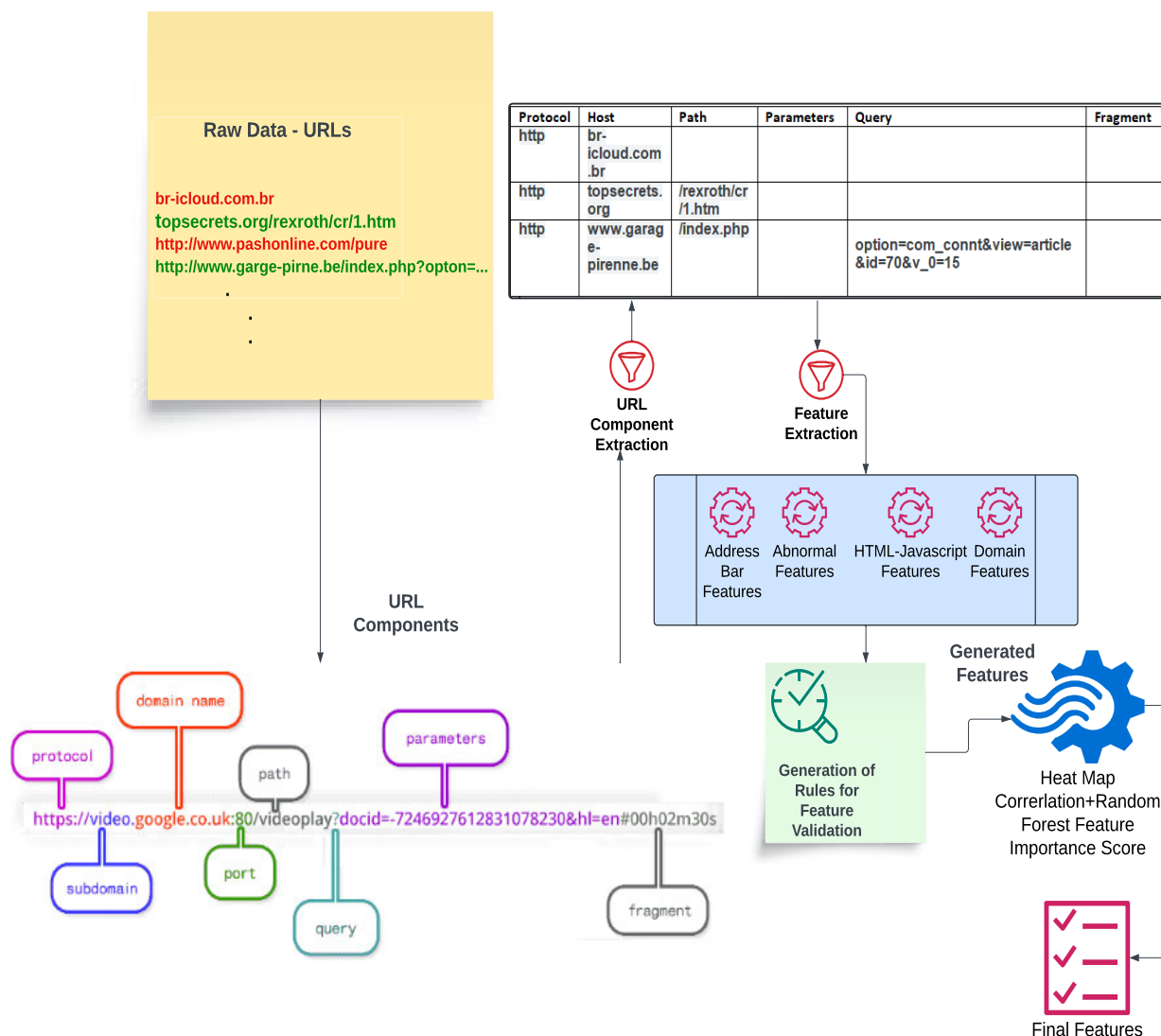


FIGURE 4. Feature extraction from URL.

rules can be interpreted and executed are shown in Tables 3, 4, and 5. The description of the URL and the features that can be extracted from it are shown in Fig. 5. The dataset is made available for reproducible purpose at [54].

a: CASE STUDY

This section provides information, describing the dataset parameters and the dataset characteristics. Since we are dealing with non numerical data, statistical analysis is not applicable. The URL is mainly formed to address web pages. The subdomain, along with path and file components, can be fully controlled by a phisher and set any value to them. The main reason security preservers struggle to detect fake domains is due to the distinctive part of the website domain (the FreeURL). If a domain has been detected as deceptive in the past, it can be used to prevent any netizens from accessing it.

URLs can be inspected in many ways. For this study, we have divided the URL into four types. The four types of features that we can extract from the URL are:

i) HTML and JavaScript-based Features: These features address image manipulation, dynamic content, and form validation of the web pages.

ii) Address Bar-based Features: These features are used to display the name or URL of the current web page. For instance, the feature using the IP address in the Address Bar is interpreted as follows:

If an IP address has an alternative to the domain name in the URL, for instance, “http://121.23.45.11/fakeDomain.html,” users need to be cautious about knowing if someone makes an attempt to steal their sensitive information. There might also be a possibility where the IP address is converted to hexadecimal code in order to mask the user, as shown in the following link:

“http://0x.0xCB.0xBA.0x52/2/gpal.ac/indexing.html”.

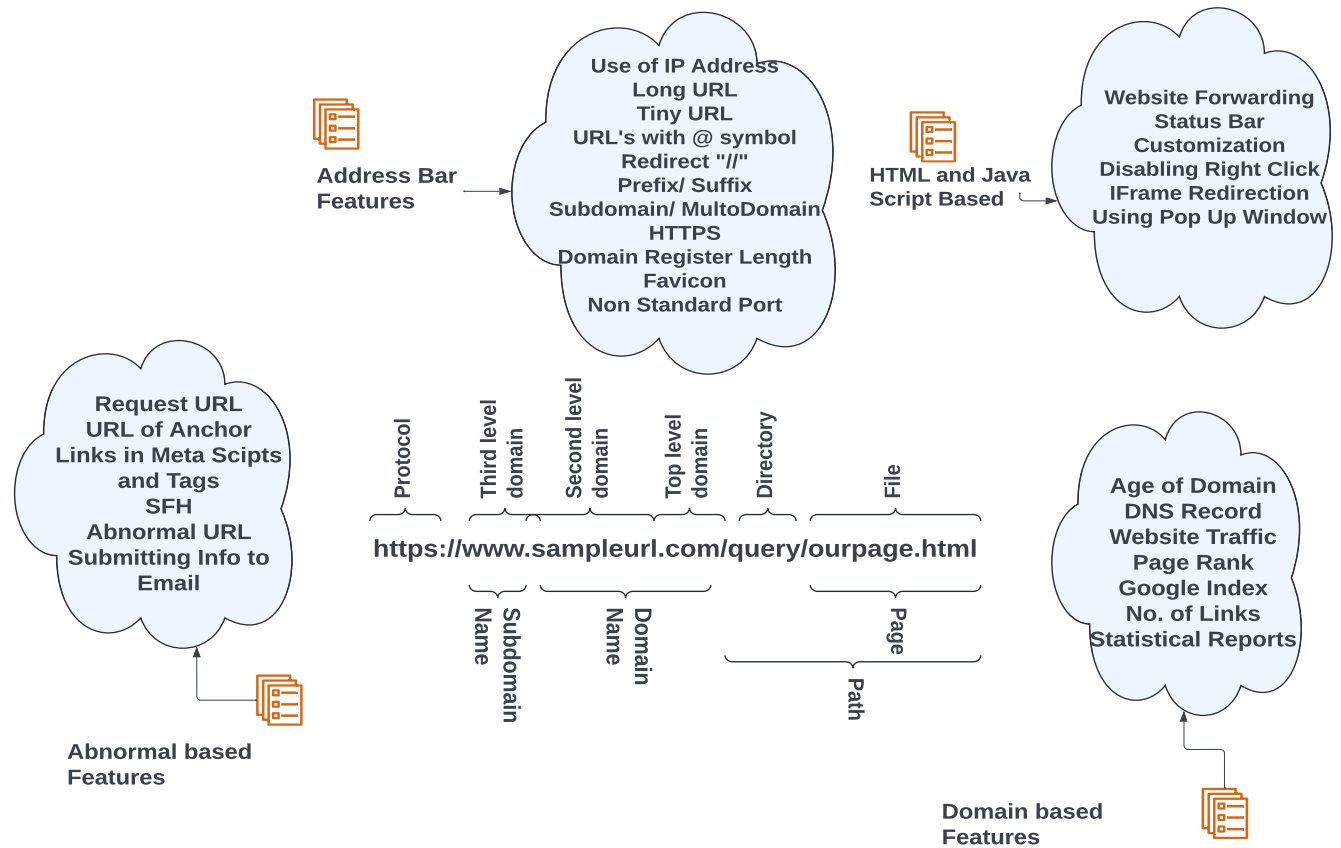


FIGURE 5. Typical URL structure.

Hence, the rule that can be applied is as below: If The Domain Part has an IP Address → Phishing Otherwise → Legitimate

Similarly, the other features shown in Tables 3, 4, and 5 are also evaluated.

iii) Abnormal-based Features: This feature checks if there are features that do not follow the standard protocols of a legitimate URL.

iv) Domain-based Features: These features explain how the company's network resources are organized.

The custom data contains 5000 rows and 64 features based on attributes like type, length of URL, number of letters, digits, special characters in the URL, shortened abnormal URL, usage of http, having only IP, root and path of the domain, and so on. The data is collected from various public sites. Legitimate URLs are collected from the latest updated sites like Alexa, and phishing URLs are collected from Phish Tank.

b: DATA PREPROCESSING

The collected URLs go through a feature extraction process. The features are compared with the standard RFC 3896, which is a generic syntax for defining a uniform resource identifier. It refers to grammar pertaining to URIs, as referred

to in Fig. 5. The attributes that have values of zero are dropped because they will not add any value to the preprocessing. The if-then rules are applied to all features by mapping them to the standard threshold values. The rules applied are depicted in Tables 3, 4, and 5 for various types of features. The optimal features are extracted using heat-map correlation along with random forest feature importance score as shown in Fig 4.

Irrelevant or non-optimal features are so-called mainly due to their less or no contribution towards the outcome. In phishing detection problems, the main goal is to accurately detect whether the URL is legitimate or fake.

For example: During the data collection phases, in our case, the URLs, there is an attribute called URL ID or index, which gives a count of the total URLs collected and also addresses each URL separately. But with respect to the outcome of detecting whether a URL is fake or not, this attribute is not necessary. Also, there are some attributes like 'path_ =', 'server_client', whose values after normalizing remain the same for all the samples. Hence, applying any kind of model to constant values will not give good results. Hence, having or not having these attributes will not alter the final classification. So avoiding these attributes can save us time from computation. Similarly, while applying various feature selection techniques, there will be weights given to each and every attribute with respect to its contribution to

TABLE 3. HTML-javascript based feature validation rules for phishing URL.

Feature Type	Parameter	Rules
HTML-Javascript Based	Favicon	If Favicon is Loaded from an external domain
	Website Forwarding	If Pages redirected ≥ 4
	Status Bar	If onMouseOver changes status bar
	IFrame Redirection	If Usage of iframe
	Right Click	If Right Click is Disabled
	Pop-up Window	If Pop-up Window Contains Text Fields

the result. So the features that contribute less to impacting the target variable will be discarded because having them or not will not impact the accuracy of the outcome. These less optimal features are termed irrelevant. In our research article, two types of feature selection techniques are used.

i) Heat map correlation: The heat map depicts the power and association of the correlation between each pair of features. If there is a high positive correlation (closer to 1), it indicates that two features are highly related. If there is a low correlation (closer to 0), it indicates that there is very little to no linear relationship between the attributes [62].

ii) Random Forest Feature Importance Score: Feature importance is a technique that calculates a score for all the input features for a given model. The scores represent the “importance” of each attribute. A higher score means that the specific attribute will have a greater effect on the model that is being used to predict a certain variable. The feature importance in Random Forest can be calculated using a metric called Gini importance, which is a measure of the total depletion of the Gini impurity of the dataset when a particular feature is used for splitting. The higher the Gini importance, the crucial the attribute is for the model [63].

For the custom dataset, which initially had a total of 64 features, after applying the above two feature selection techniques, the features were reduced to 50 without much impact on accuracy or other performance metrics.

B. EXPERIMENTAL SETUP

The experiment is carried out in Google Collab software. The libraries and framework used are in the Python language. The

system contains 16 GB of RAM, a 64-bit operating system, an x64-based processor, and the Windows 10 operating system.

C. MODES OF PROCESSING

The main intention of the research is to focus on how efficiently data gets processed in various modes of execution. The data sets are processed in three modes:

i). Offline mode:

In offline mode, the data is collected in advance. The data is then fed as input to the model for processing. The advantage of using this mode of execution is that data is available at any time interval and can be traced back for evaluation. The limitation of using this model is that real-time processing of data cannot be done. Hence, real-time applications that require on-the-go processing of data cannot be performed. Also, storage problems arise when the size of data increases.

ii). Batch Mode:

This mode of execution is performed in batches. The batch size has to be specified during the coding phase. The data gets processed in the batches specified. The main advantage of this kind of processing mode is that one can address storage issues because data will be processed in batches and rerun with new batches. The limitation of this approach is the time delay in the collection of data to fill in the batch size and obtain the desired result.

iii). Incremental Mode.

In online or incremental processing mode, data is processed as and when fed. The advantage of this mode of processing is that data is processed as and when it arrives

TABLE 4. Address-bar based feature validation rules for phishing URL.

Feature Type	Parameter	Rules
Address bar	IP Address	If IP address is used as a substitute for name of the domain
		If IP address is transformed to hexadecimal code
	Path	IF URL >= 2048 characters
		If issuer and https Is Trusted and Age of Certificate < 1 year
		If Tiny URL exists there is a possible redirection to fake website
		IF "http" is present instead of "https"
	Special Characters	If @ symbol present in URL it ignores everything preceding to it
		If // symbol means user will be redirected to other website
		If dash(-) symbol is present makes URL look legitimate
		Similarly "= . # % + \$! * , " special symbols are interpreted

without any delay, which is very useful for real- time applications. The drawback of this mode is that data is available to process only once and cannot be traced back. Hence, sufficient care needs to be taken to process the data without any errors. The following section explains the frameworks used for the various modes of execution, the techniques used, and the flow of data for the research.

Some of the features of different modes of processing are:

1. In offline mode, data needs to be available beforehand to apply pre-processing techniques like cleaning, transformation, or applying relevant models. The data is collected, loaded, preprocessed, and fed to the models for classification. The appropriate classification model fits the processed data. The Python framework and applied classification models are used on the Pandas data frame. The goodness of the model is measured using various performance metrics.
2. In batch mode, data is processed in chunks or batches for classification. For the batch processing, Python’s PyTorch and PySpark frameworks are used. The batches are divided in the sizes of 32, 64, 128 and so on. The obtained outputs are then compared and analysed.

3. For the incremental processing, Python’s River framework is applied. Data is processed in dicts, which is a Jason-like format used for fast processing.

The phishing data set makes use of machine learning classifier models like random forest, extra trees, hoeffding tree, approximate large margin algorithm (ALMA), adaboost, gradient boost, K nearest neighbor, support vector, logistic regression, linear discriminant analysis (LDA), decision tree, adaptive random forest (ARF), and naive bayes (bernoulli) and multinomial for the classification of websites as legitimate or phishing. Deep learning models like Multi-Layer Perceptron (MLP), Binary Neurel Network (BNN), and Keras Sequential models were also applied. The proposed architectural flow of the process which depicts the data preprocessing followed in the research along with machine learning model building and execution modes is shown in Fig.6.

The steps followed are as follows:

Step 1: The input data is fed to the preprocessor. The preprocessor makes sure the data is cleaned by checking for null or missing values and substituting values wherever

TABLE 5. Abnormal based feature validation rules for phishing URL.

Feature Type	Parameter	Rules
Abnormal and Domain Based	Links	If no Host name in URL
		If % of Links in Meta Script and Link $\geq 81\%$
		If Number of Links Pointing to Page > 3
	Request URL	If Request URL $\geq 61\%$
	Anchor URL	If % of URL Of Anchor $\geq 67\%$
	Ports	If Port number not standard
	SFH	IF SFH is "about: blank" Or Empty
	Email	If mail() or mailto: function is used to submit user info
	Digits counts	If characters are replaced by digits in a domain name
Domain Based	Age	If Domain Expires on ≤ 1 years
		If Domain age ≤ 6 months
	Sub-domain	If domain dots > 2
	DNS Record	If absence of DNS Record for Domain
	Website Rank	If Rank of Website $> 100,000$
	Page Rank	If Rank of a Page < 0.2
	Google Index	If Webpage is not Indexed by Google
	Statistical-Reports	If Statistical reports show that Host belongs to Top Phishing IPs or Domains

necessary. It also checks if the data is balanced, normalized, and ready to process. The target values need to be balanced to train the models and get appropriate results. The data is normalized or standardised to maintain uniformity among different attribute values. Feature extraction is done to optimise the model to focus on only the relevant features.

Step 2: The data from the preprocessing steps is fed to the model building, where the data is divided into training and testing in the ratio of 70:30 for the model building. The machine learning classifiers are applied to the model to build it. To avoid potential bias, custom data is also processed.

Step 3: The model building is carried out in three different modes (online, batch, and incremental) of execution to compare its performance. The anomaly scores and various performance evaluation metrics are compared and analysed.

The experiment is conducted with three datasets to avoid any potential bias. The phishing dataset has many attributes and hence needs feature extraction during data preprocessing. The SMS spam ham dataset has only two attributes. Hence, the feature extraction process will be eliminated, but processes like tokenization, removing stop words, and vectorization will be applied as part of data preprocessing.

IV. ANALYSIS AND RESULTS

Some of the analysis, insights, and key findings observed from processing data in various modes are as follows:

1. By processing data in offline (static) mode, the computing system would crash if the data exceeded the computing capacity. However, the temporary solution could be to use optimised data types or use sample data to save

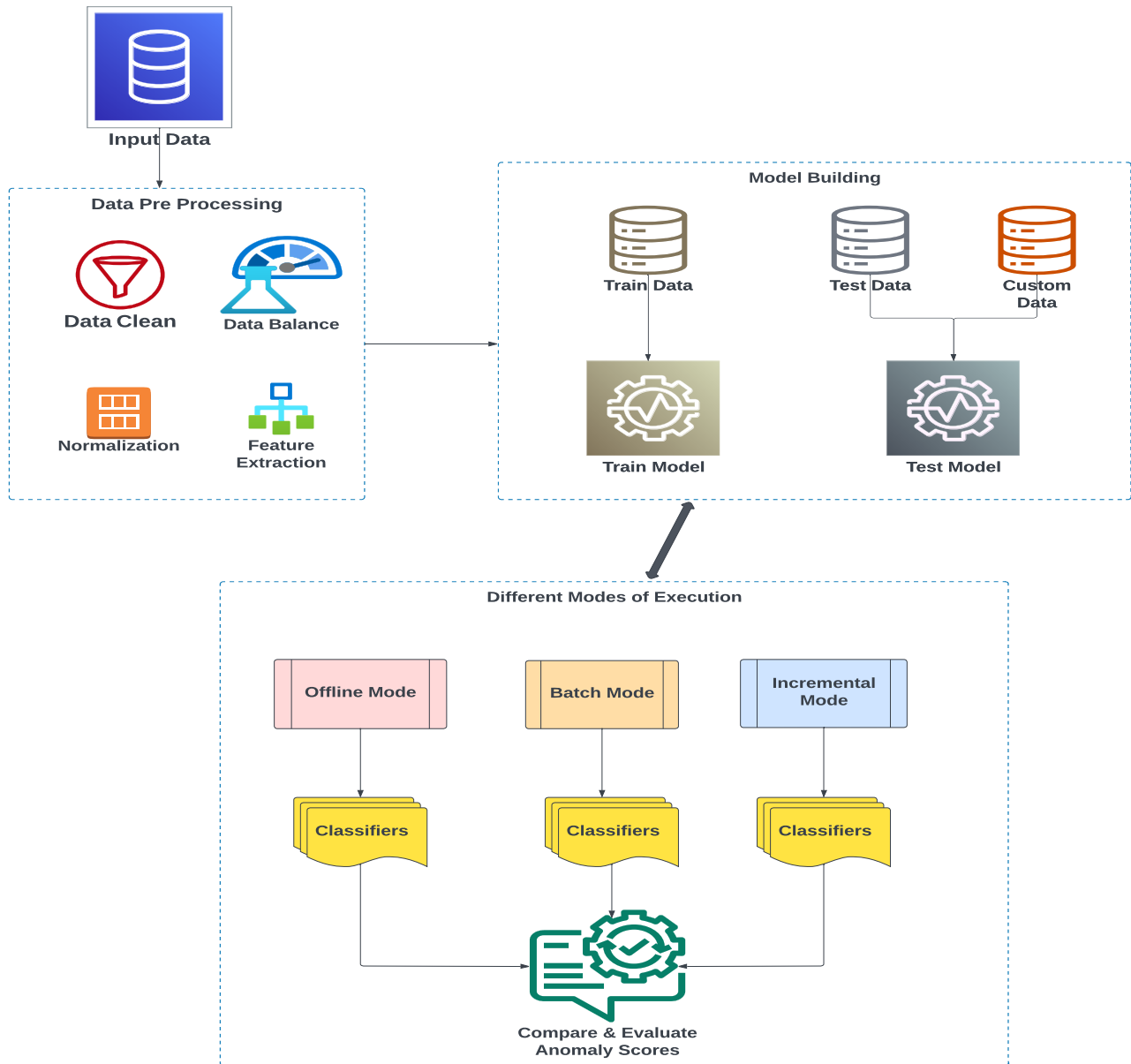


FIGURE 6. Proposed architectural diagram.

the processor; however, this would not lead to accurate results.

2. One needs to choose appropriate hardware to handle big data. However, including such hardware would lead to a cost burden. A possible solution could be to process data in chunks (mini-batches) like Dask and Spark's MLib [21], PySpark, or PyTorch. Another critical drawback of batch learning or static data is that it does not learn from new data. As and when new data arrives, the model has to learn from scratch, computing old and new data and creating a new data set. This leads to annoying situations where data flows in every week, day, hour, minute, and even seconds in a real-time situation.

3. A feasible approach would be to process data incrementally as and when it arrives, like from the river framework [23]. It addresses the storage and computing problems of the

traditional approaches. There are also various methods in the river framework that can handle concept drift.

Tables 6 and 7 give a summarised overview of the processing of two datasets, the Mendley Phishing dataset and the Custom Phishing dataset, on the three different modes of execution, such as offline, batch and incremental using various classifiers. The pros and cons of each classifier are also justified with respect to the phishing dataset.

A. ANALYSIS AND RESULTS FOR THE MENDLEY PHISHING DATASET

In the following section, the results obtained by applying machine learning and deep learning algorithms to various modes of execution are shown.

TABLE 6. Various machine learning techniques applied on phishing data sets.

Modes of Execution	Dataset Used	Classifiers Applied	Pros	Cons
Static (Offline)	Phishing Mendley[53] Custom[55]	Random Forest	Handles non-linear, High Dimensional data	Acts like black box approach for a statistical modelers
		Extra Trees	Reduction in bias	Overfitting
		Multi Layer Perceptron	Effective for chaotic time series modeling	Complex to optimize due to non linearity and unpredictability
		AdaBoost	Handles noisy data	Requires high quality data
		XGBClassifier	Do not need Normalized features	Overfitting
		Decision Tree	Interpretatibilty, Less data preparation	Overfitting
		Gradient Boosting	Predictive Accuracy	Computational Expense
			Handles missing data	Overfitting
		Support Vector Machine	Versatile	
			Memory Efficient	Difficult with Big Data
		K Nearest Neighbors	Parallel Execution	Cannot handle imbalanced Data
			Non Parametric	Sensitive to k
		LDA	Supervised Dimensional Reduction	Singularity Problem
		Naive Bayes	Suits Text Classification	Not suitable for context switch
Batch	Phishing Mendley[53] Custom[55]	Multi Layer Perceptron	Effective for chaotic time series modeling	Complex to optimize due to non linearity and unpredictability
		Binary Neural Network	Efficient with low power devices	Causes Information Loss
		RNN	Extensive Memory	cannot keep track long-term dependencies
			Greater Accuracy	
		Logistic Regression	Easy to set up	Cannot predict continuous data
		Random Forest	Handles non-linear, High Dimensional data	Black box approach for a statistical modelers
		Decision Tree	Interpretatibilty, Less data preparation	Overfitting

TABLE 7. Various machine learning techniques applied on phishing data sets(contd).

Modes of Execution	Dataset Used	Classifiers Applied	Pros	Cons
Incremental (Online)	Phishing Mendley[53] Custom[55]	K Nearest Neighbor	Parallel Execution	Cannot handle imbalanced Data
			Non Parametric	Sensitive to k
		Approximate Large Margin Algorithm	Balance Data Online	Permissive Distribution
		HoeffdingTree		Desired
			Compares features better than others	Assume distribution of data as stationary
		Logistic Regression	Easy to set up	Cannot predict continuous data
		Adaptive Random Forest	Adapts to concept drifts	Computational Complexity
		Perceptron	Effective for Linear seperable problems	Does not suit real world scenarios
		Extremely Fast Decision Tree	Constructs itself incrementally	Slower to process data
		Hoeffding Adaptive Tree	Uses drift detector to increase accuracy	Only support an instance-by-instance learning
		Half Space Tree	Works well when anomalies are spread out	Model misses anomalies early on in the data stream

TABLE 8. Evaluation metrics on offline mode for phishing dataset.

Model Accuracy and Loss with all features			
Models	Accuracy	CV Errors	Time (secs)
Random Forest	96.98	0.0031	1
Extra Trees	96.87	0.0029	2
MLP	96.41	0.0021	2
AdaBoost	95.83	0.0018	1
XGBClassifier	96.56	-	2
Decision Tree	95.54	0.0063	1
Gradient Boosting	94.77	0.0054	2
SVC	94.55	0.0042	1
KNeighbors	93.53	0.0040	2
Logistic Regression	92.90	0.0046	1
LDA	92.12	0.0057	2
Naive Bayes	60.91	0.0036	2

TABLE 9. Evaluation metrics on offline mode for phishing dataset.

Model Accuracy and Loss removing irrelevant features			
Models	Accuracy	CV Errors	Time (secs)
Extra Trees	96.68	0.0025	2
Random Forest	96.79	0.0012	1
MLP	95.90	0.0035	2
Decision Tree	95.44	0.0028	1
AdaBoost	95.26	0.0013	1
XGBClassifier	97.12	-	2
Gradient Boosting	94.52	0.0040	1
SVC	94.39	0.0035	1
KNeighbors	93.50	0.0016	1
Logistic Regression	92.99	0.0038	1
LDA	91.93	0.0031	1
Naive Bayes	60.32	0.0023	1

1) OFFLINE MODE

The phishing dataset is processed in offline mode with various classifiers. Offline mode refers to data that is

collected beforehand. The data is then fed to the models after pre-processing. Tables 8 and 9 depict the classifiers applied

TABLE 10. Evaluation metrics on batch mode (PyTorch) with all features for phishing dataset.

Model	Precision	Recall	F1 Score	Accuracy	Time	BS
MLP	0.97	0.98	0.97	96.82	370	64
MLP	0.96	0.98	0.97	96.56	250	128
Binary NN	0.97	0.97	0.97	97	350	5
RNN	0.92	0.95	0.94	93.03	2520	16

TABLE 11. Evaluation metrics on batch mode (PyTorch) excluding irrelevant features for phishing dataset.

Model	Precision	Recall	F1 Score	Accuracy	Time	BS
MLP	0.96	0.98	0.97	96.38	360	64
MLP	0.96	0.97	0.96	95.77	240	128
Binary NN	0.97	0.94	0.95	96	300	5
RNN	0.93	0.94	0.94	93.85	2105	16

TABLE 12. Evaluation metrics on batch mode (PySpark) for phishing dataset with all features.

Model	Accuracy	time (secs)	Precision	ROC	Recall	BS
LogisticRegression	92.92	55	0.93	0.98		
Random Forest	97.18	680	0.97	0.99		
GBTClassifier	96.44	1700	0.96	0.99		
Decision Tree	95.92	70	0.95	0.94		
MLP	93.68	26	0.94		0.94	64
MLP	93.58	28	0.94		0.94	128

TABLE 13. Evaluation metrics on batch mode (PySpark) for phishing dataset without irrelevant features.

Model	Accuracy	time (secs)	Precision	ROC	Recall	BS
LogisticRegression	92.12	53	0.92	0.97		
Random Forest	96.10	660	0.96	0.99		
GBTClassifier	91.84	1800	0.96	0.98		
Decision Tree	95.36	60	0.92	0.94		
MLP	94.90	22	0.94		0.94	64
MLP	94.95	13	0.94		0.94	128

to the phishing dataset and the accuracy and cross-validation errors (CV errors) obtained for the different models for all features.

The evaluation metrics and execution time obtained before and after eliminating irrelevant features are also depicted. The aim is to choose optimal attributes for the process. It can be noted that random forest and extra tree classifiers give better results for the phishing dataset. The random forest

builds multiple decision trees over subsets of data that are bootstrapped, whereas extra trees build multiple decision trees over the entire dataset. The Naive Bayes classifier performs poorly, as shown in Fig. 7. This is because of the property of Naive Bayes that it assumes all the attributes are mutually independent, which is not true for the phishing website data set that has been considered for the study. There is also no significant drop in accuracy even if nine

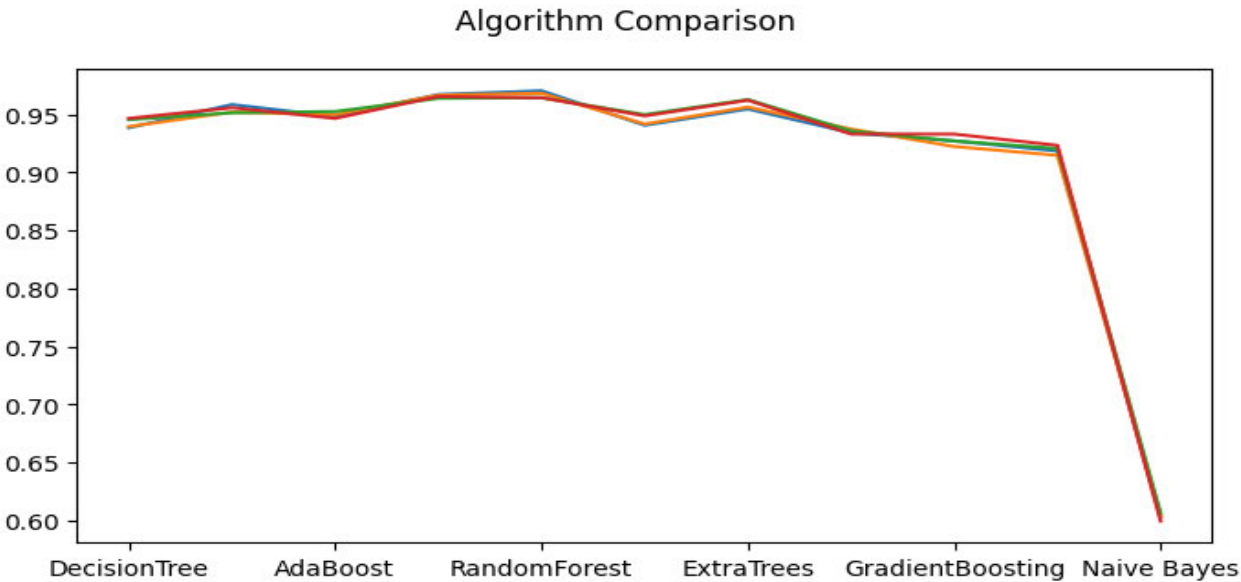


FIGURE 7. Algorithm comparison.

TABLE 14. Evaluation metrics on online mode (A).

Model accuracy with all features for Phishing dataset		
Model	Accuracy	time (secs)
KNN	90.64	150
ALMA	86.79	1
HoeffdingTree	89.87	4
Logistic Regression	92.17	2
Perceptron	90.60	2
Extremely Fast DT	89.90	6
Hoeffding Adaptive Tree	85.95	5
Half Space Tree	45.28	3
Adaptive Random Forest	96.61	26

TABLE 15. Evaluation metrics on online mode (B).

Model accuracy without extra features for Phishing dataset		
Model	Accuracy	time(secs)
KNN	91.22	130
ALMA	88.3	1
HoeffdingTree	90.50	3
Logistic Regression	92.33	2
Perceptron	90.68	1
ExtremelyFastDT	90.13	5
HoeffdingAdaptiveTree	85.95	4
Half Space Tree	45.60	1
Adaptive Random Forest	96.82	20

attributes are removed from the dataset. Hence, eliminating unnecessary attributes can reduce computing time and storage issues.

2) BATCH MODE

In batch mode, the data is processed in chunks due to the high volume of data and the nature of the task being repetitive. The batch processing is conducted using two frameworks. First is PyTorch, which is a deep learning Python library for batch processing and is ideal for research due to its flexibility and quick editing capabilities. It can run on GPUs for faster execution. PyTorch’s DataLoader module divides the training data into batches, which are later processed sequentially. The

second framework used is PySpark, which is the Python API. It is an open-source, distributed computing framework mainly used for real-time and large-scale data processing by processing data in parallel. The Spark session builder module assists in creating batches for processing the data. The size of the batch is specified during the coding phase. Tables 10, 11, 13, and 14 show a detailed overview of mini batch processing using PyTorch and PySpark. The evaluation metrics, such as precision, recall, F1 score, and accuracy, are calculated. The Random Forest classifier performs better compared to other classifiers. The time taken by the machine learning algorithms to run the process and the batch sizes are also specified in the table.

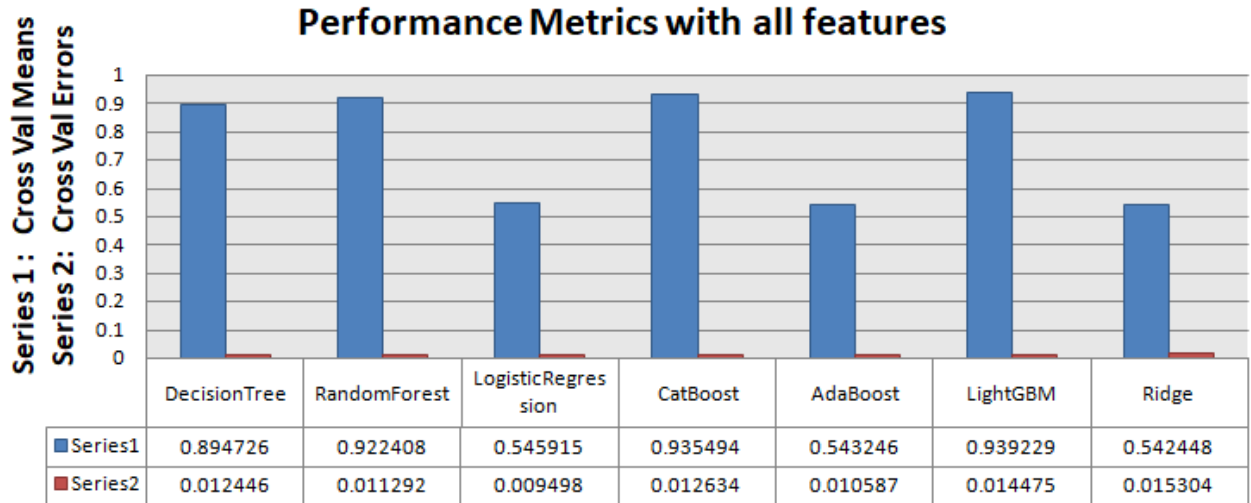


FIGURE 8. Metrics evaluation on custom data in offline mode with all data.

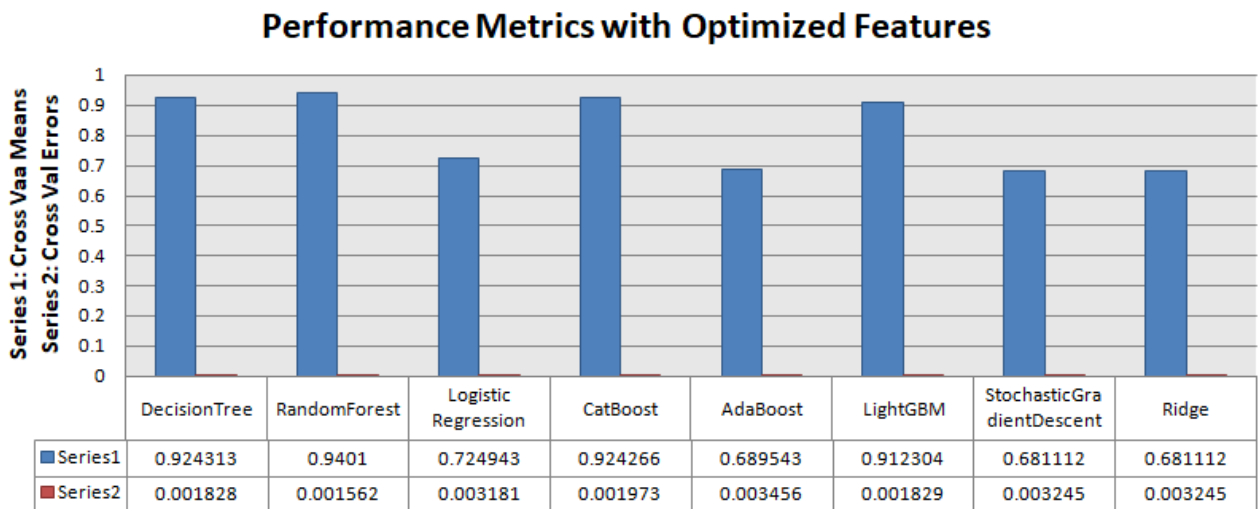


FIGURE 9. Metrics evaluation on custom data in offline mode with optimized data.

3) ONLINE MODE

For the incremental mode, experiments were conducted using the River framework, which is a Python framework. All the tools in the river library can be updated with a single observation in time, making them suitable for streaming data. The stream processing uses incremental processing. The experiments were conducted with real and synthetic data sets to understand, compare, and evaluate the space and time complexity. Hence, our effort is to modify the existing algorithm by prioritising the crucial features, applying the algorithm, and comparing the outcomes. The River framework also supports concept drift, which is crucial in real-time applications. Tables 14 and 15 give the performance evaluation of the Mendley phishing dataset on machine learning models with all features and optimised features, respectively.

B. ANALYSIS AND RESULTS FOR CUSTOM DATASET

The following sections give an analysis of the custom data processed in various modes of execution. Custom data is experimented with mainly to eliminate any kind of potential bias that could persist in publicly available datasets. The custom dataset consists of 5000 URLs collected from various sources. There were about 64 features in the data. The URLs were preprocessed, and feature extraction techniques were applied to choose the optimal features. The heat map correlation and random forest feature importance score techniques were used to filter the necessary features. After the feature extraction process, the total features were reduced to 25.

1) OFFLINE MODE

Figures 8 and 9 plot the performance metrics of custom data on machine learning models. There is a slight improvement

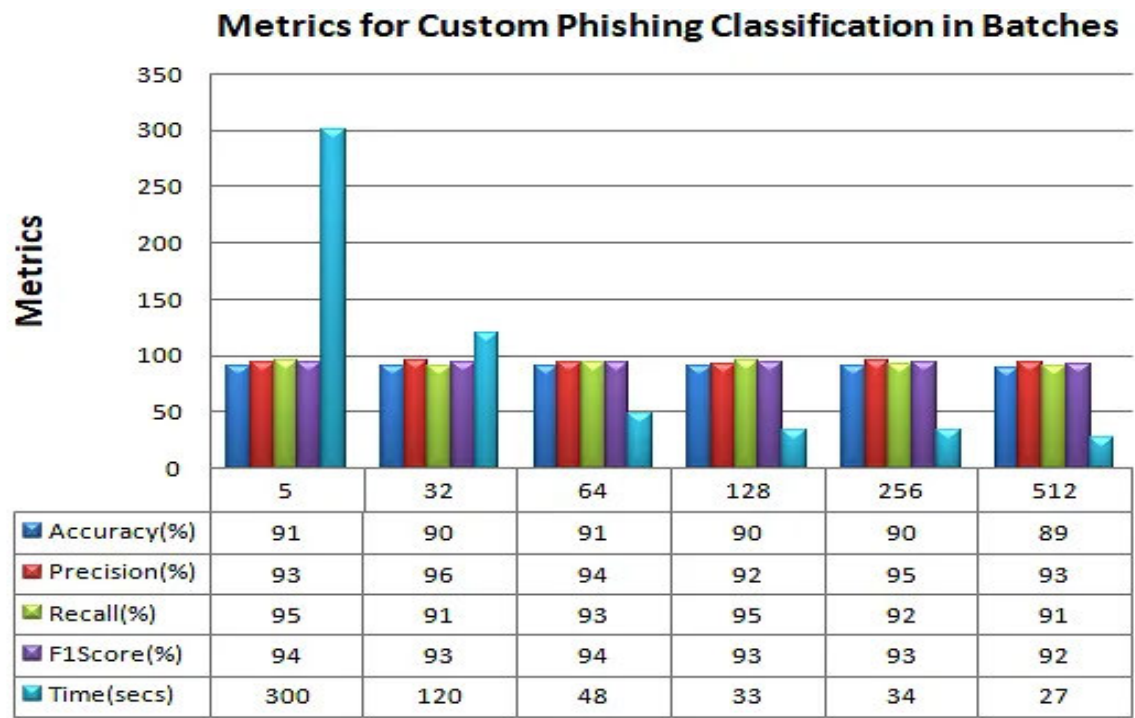


FIGURE 10. Metrics evaluation on custom data in batches using PyTorch.

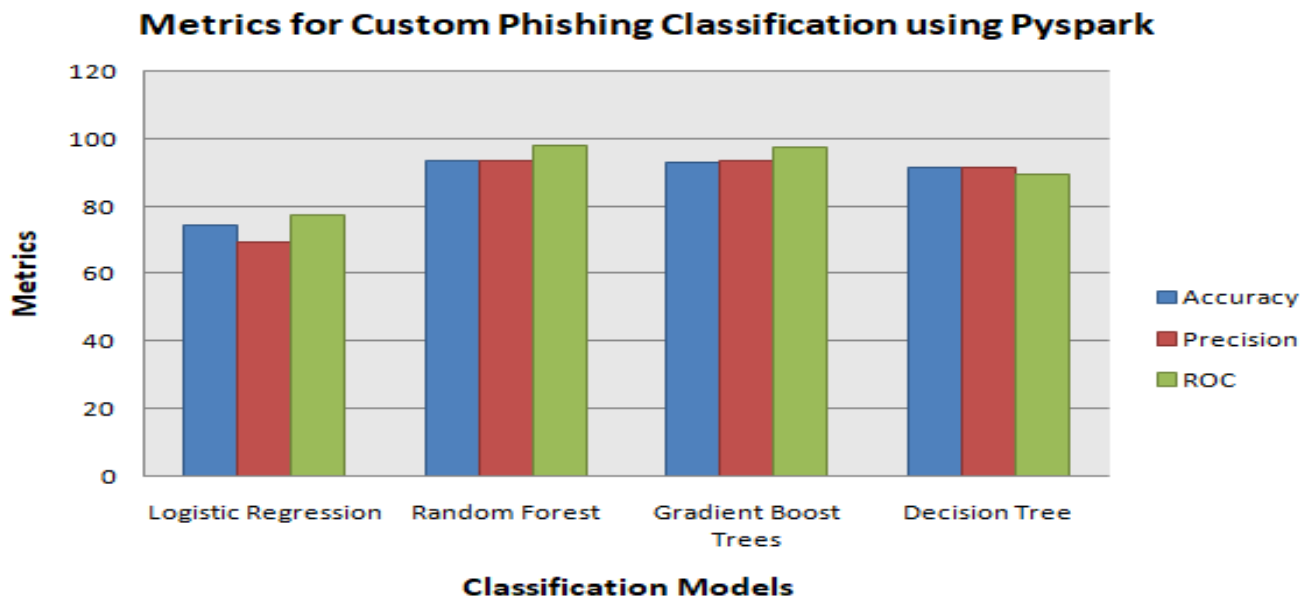


FIGURE 11. Metrics evaluation on custom data using PySpark.

in performance and time complexity when included with optimised features for the experiment.

2) BATCH MODE

Batch processing evaluates the data in various batches. To understand the impact of various batch sizes on the data, experiments were conducted with varying batch sizes. The

model used was a binary neural network model with batch sizes of varying sizes, such as 5, 32, 64, 128, 256, and 512, using the PyTorch Framework. As seen in Fig.10, the analysis obtained shows that as the batch size increases, the time taken to execute decreases. The other metrics, like accuracy, precision, recall, and F1 score, do not vary to a greater extent. The data set was also executed using Pyspark

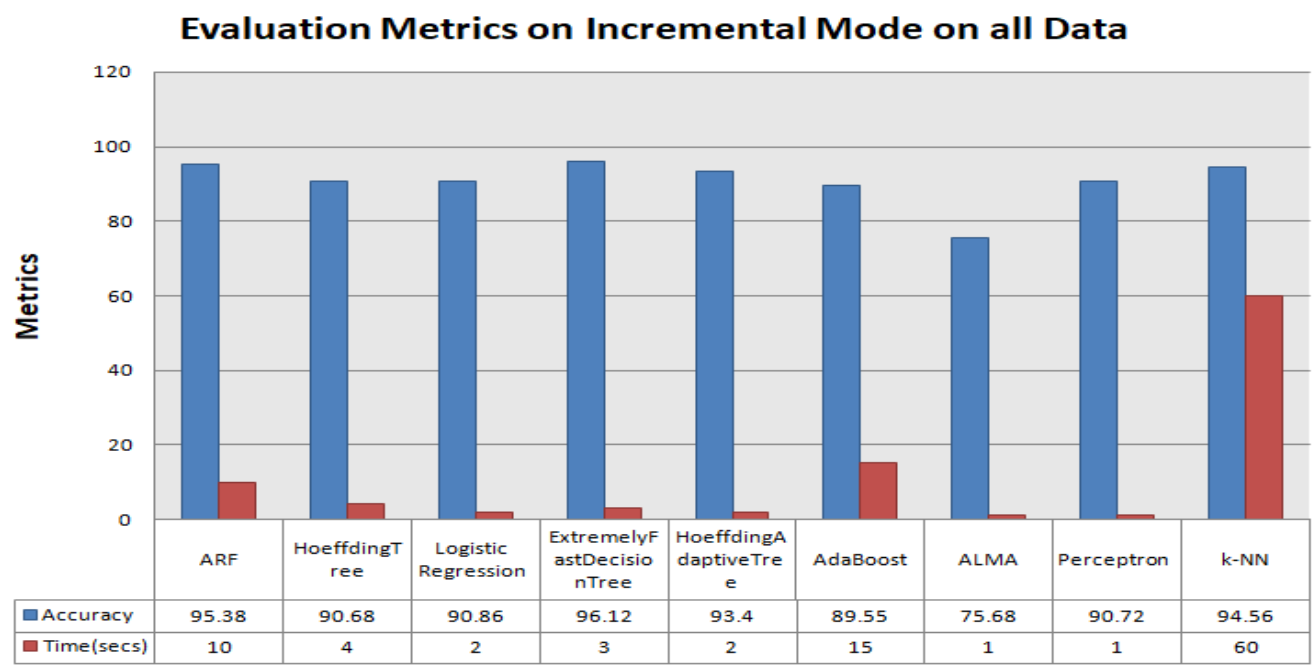


FIGURE 12. Metrics evaluation on custom data with all features.

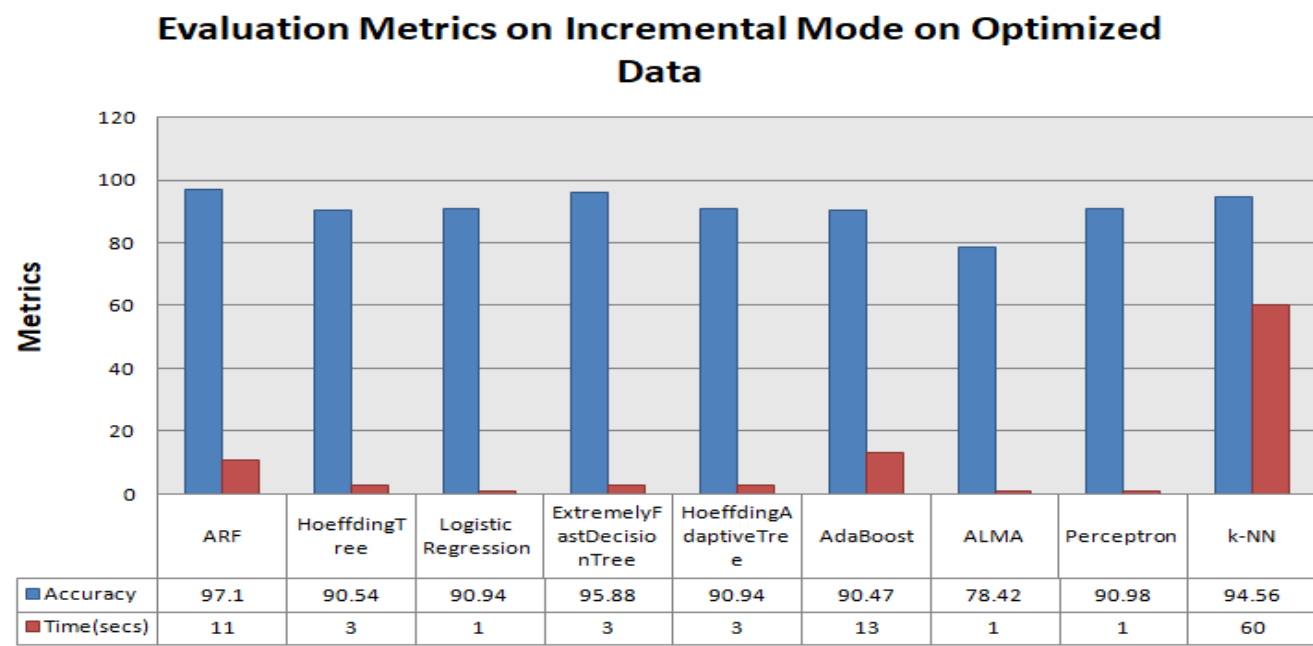


FIGURE 13. Metrics evaluation on custom data with optimised features.

on classifiers like logistic regression, random forest, gradient boost trees (GBT), and decision trees. The Random Forest produced better results in terms of evaluation metrics. Fig.11 shows the results. Logistic Regression and Decision Tree took 53 and 35 seconds to execute, respectively, whereas Random Forest Classifier and GBT Classifier took a longer time, 780 seconds and 1740 seconds, respectively. The main

reason being the property of random forest to create the trees from the beginning as and when it notices a concept drift.

3) INCREMENTAL MODE

The incremental mode was executed using the River Framework. The River Framework is an upcoming framework gaining popularity due to its various features in

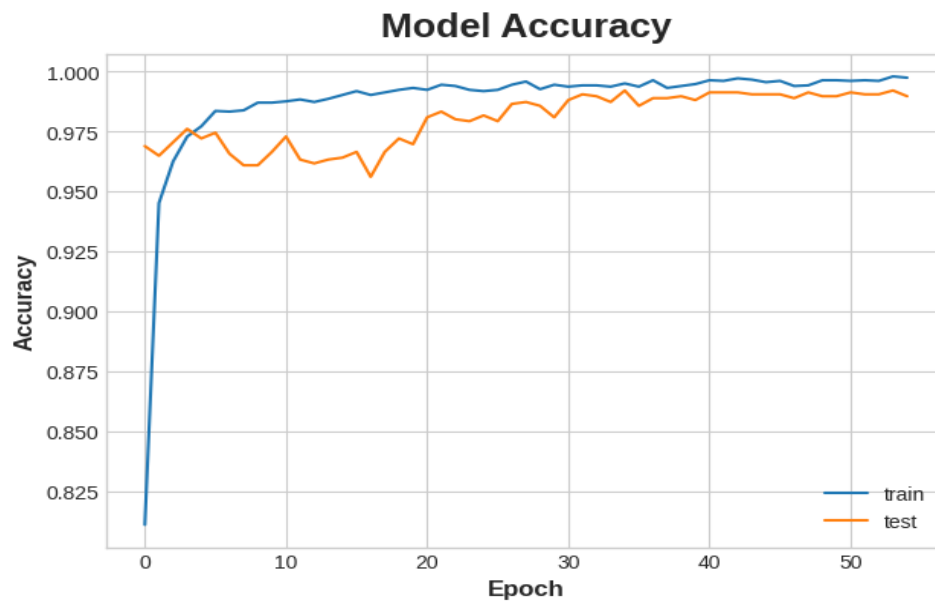


FIGURE 14. Accuracy evaluation on custom data with all features.

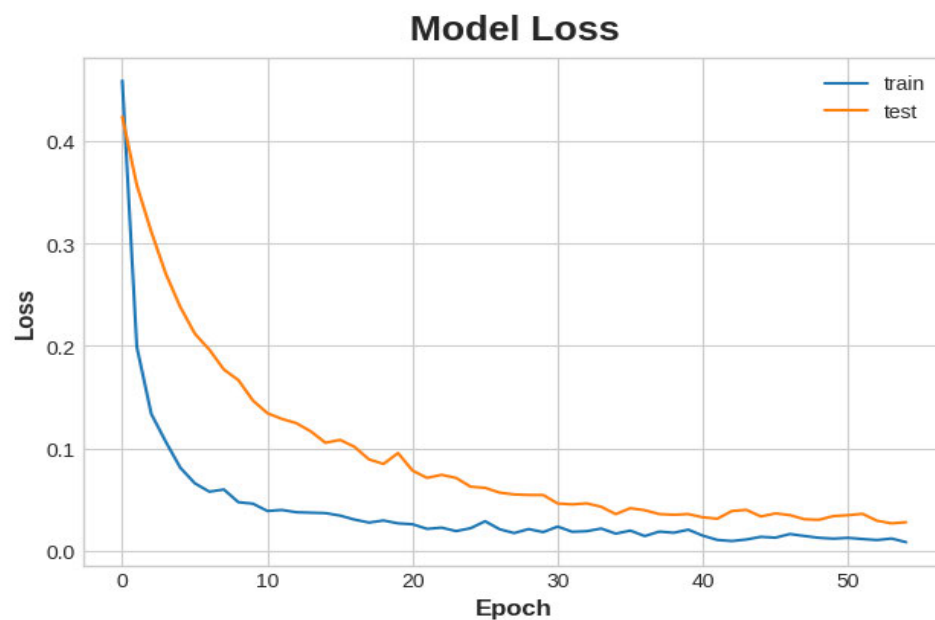


FIGURE 15. Loss evaluation on custom data with all features.

handling streaming data. It can also handle concept drift. Due to this feature, the River framework can be used for processing data with numerical attributes. Since the framework uses dictionaries to process data, like in the Jason format, computing becomes faster without requiring any additional storage. Fig. 12 shows the performance metrics on custom data with all the data. Fig. 13 shows the data executed in the River framework with optimised features.

C. ACCURACY AND LOSS EVALUATION

To show the potential overfitting issues that existed in the data, the training process was conducted to identify and plot the accuracy and loss charts on the custom dataset. This would provide a clearer understanding of the model's performance over time. The training process was detailed with accuracy and loss charts for a custom dataset by including all and optimised features using a Keras sequential model with 200 epochs. Two layers of batch normalization

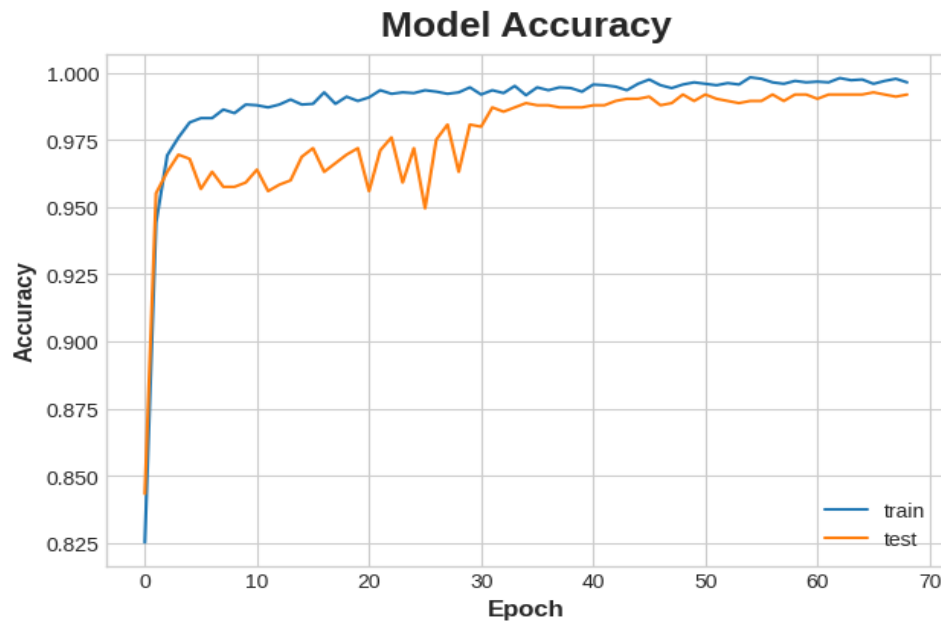


FIGURE 16. Accuracy evaluation on custom data with optimised features.

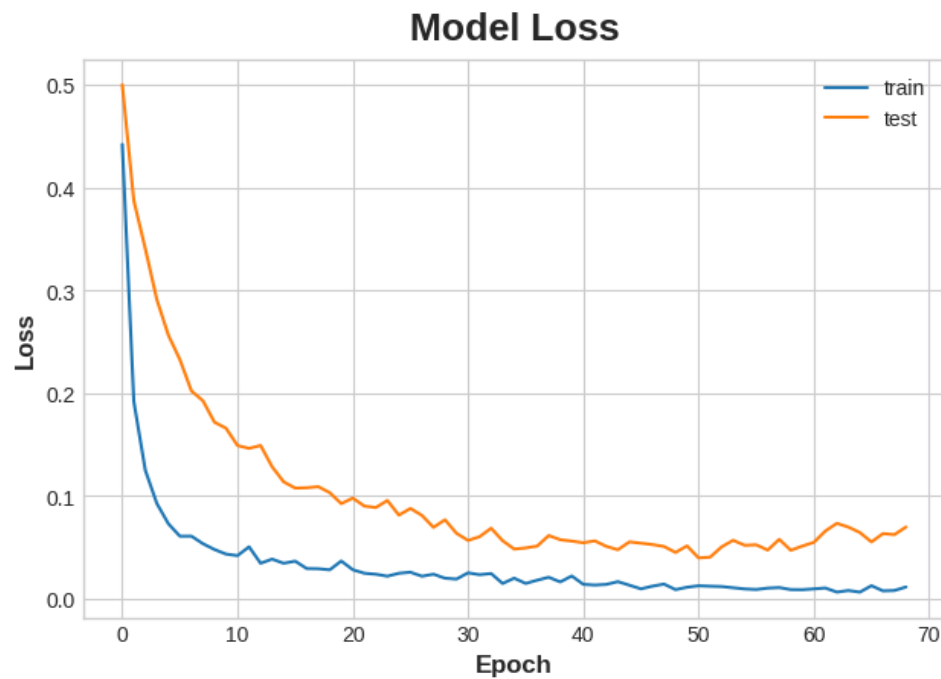


FIGURE 17. Loss evaluation on custom data with optimised features.

with 0.3 values as dropouts were used. The activations used were relu and sigmoid. The model was compiled with Adam Optimiser, and the loss metric used was binary cross-entropy. The accuracy metric used was binary accuracy. The best test loss obtained was 0.0268, and the best validation accuracy obtained was 99.20% on the data with all 64 features included. With the heat map and Random Forest feature

importance score feature extraction applied to the dataset, the best test loss obtained was 0.0397 and the best validation accuracy obtained was 99.28% on the data with 50 features included. Figures 14 and 15 show the model accuracy and loss evaluation on a custom dataset with all features. Similarly, Figures 16 and 17 depict the model accuracy and loss evaluation on a custom dataset with optimised features.

TABLE 16. Evaluation metrics on offline mode.

Models	Accuracy	Precision	Recall	F1 Score	Time(secs)
MultinomialNB(TFIDF)	96.53	0.96	0.73	0.84	1
MultinomialNB(MinMax, TFIDF)	98.27	-	-	-	2
MultinomialNB(UnScaled, TFIDF)	87.62	-	-	-	1
MultinomialNB Pipeline*	96.65	-	-	-	15
KNN	93.64	-	-	-	40
KNN Pipeline	95.16	94.82	0.61	0.76	80
SVC Pipeline	98.39	98.19	0.87	0.93	240
SGD Pipeline	98.80	98.79	0.91	0.95	150
Gradient Boosting	97.37	97.46	0.82	0.88	180
XGBoost	96.41	96	0.74	0.84	360

D. ANALYSIS AND RESULTS FOR SMS SPAM/HAM DATASET

SMS data are categorised as spam for unsolicited messages and ham indicating legitimate messages. The SMS dataset undergoes data preprocessing steps like tokenization, stemming, and lemmatization, thereby applying pipelining. Pipeline means applying the classifier model, countvectorizer to remove punctuation and stop words, and TfidfTransformer.

Tables 16, 17, and 18 show the analysis done on the SPAM-HAM dataset and the relevant classification models applied to it. The performance metrics such as accuracy, precision, recall, F1-score, and time complexity are summarized. It is evident that Naive Bayes classifiers like multinomial and bernoulli give better results compared to other algorithms.

Bernoulli Naive Bayes works better in incremental processing in the river framework mainly due to the feature of Naive Bayes that it adapts quickly to changes and new data compared to random forest mainly, due to the fact that random forest has to rebuild the forest every time something changes and is dependent on all features whereas naive bayes is independent of other attributes. In SMS spam ham data, since we do not have many attributes, naive bayes works better. Also, random forest may not be well suited for the SMS dataset because it is not suitable for sparse data. Since we have only 2 variables in the SMS spam ham dataset, namely the message and the target attribute, random forest will not give optimum results.

E. KEY FINDINGS AND COMPUTATIONAL EFFICIENCY

The following section details the key findings and the inference speed or computational efficiency in real-time settings, especially for wearable or resource-constrained devices. Inference efficiency is critical for practical applications of phishing detection. State-of-the-art comparison of feature selection criteria and techniques applied are detailed in Tables 19 and 20. The main goal of our study

TABLE 17. Evaluation metrics on batch mode(PySpark).

Models	Accuracy	Precision	Recall	F1-Score	Time (secs)
MultinomialNB	98.55	0.76	0.98	0.86	3
RandomForest	95.90	1	0.69	0.82	15
Logistic Regression	98.26	0.99	0.88	0.93	10
GBTCClassifier	96.34	0.85	0.88	0.87	400
Decision Tree	96.60	0.86	0.90	0.88	17

TABLE 18. Evaluation metrics on incremental mode.

Models	Accuracy	Time(secs)
ARFClassifier	88.28	6
HoeffdingTreeClassifier	87.58	8
Logistic Regression	86.60	3
ExtremelyFastDecisionTreeClassifier	87.96	1
HoeffdingAdaptiveTreeClassifier	88.28	18
AdaBoostClassifier	87.69	28
ALMAClassifier	89.43	1
BernoulliNB	97.86	60
Perceptron	86.83	1

is to provide a detailed comparison of the execution of the phishing detection problem considering the categorical custom and publicly available phishing data sets using batch frameworks and incremental modes. This enables end users in real life who use time-streaming data to adopt incremental frameworks when the data has to be processed in real time.

TABLE 19. Comparison of various feature selection methods and classifiers on phishing data sets.

Article & Year	Dataset	Feature Selection Method	Classifiers	Total Features	Accuracy (%)
Alazaidah R et.al[34] 2024	Mendley	InfoGainAttribute	RandomForest	30	96.58
Alazaidah R et.al[34] 2024	UCI	GainRatioAttribute	FilteredClassifier, J-48	9	92.35
Abdul Samad et.al[36] 2023	UCI	selectKbest	XGB	30	98.21
Abdul Samad et.al[36] 2023	Mendley	selectKbest	GB	48	98.27
Karim A et.al[38] 2023	Kaggle	Nil	Hybrid LSD	33	98.12
Babagoli M. et al.[39] 2019	UCI	DT+Wrapper	meta-heuristics (HS, SVM)	30	92.80
Jain A. K. et al. [35] 2020	Phish Tank, Alexa	URL Features	Modified TF-IDF	14	89
Rao R. S. et al. [41] 2020	Alexa, PhishTank	Hybrid	TwinSVM	17	98.05
van Geest et. al [45] 2024	PILWD-134 K	URL, HTML, DOMtree	Hybrid Models	-	97.44
Subasi A. et al [42] 2017	UCI, WEKA	Data mining	Random Forest	30	97.36

TABLE 20. Comparison of various feature selection methods and classifiers on phishing data sets (contd).

Article & Year	Dataset	Feature Selection Method	Classifiers	Total Features	Accuracy (%)
Suleman M.et al. [43] 2019	UCI	Automatic selection by GA	Dichotomiser-3 Genetic Algorithm(GA)	15	95
Erzhou Zhu et. al [47] 2024	PhishTank,Alexa Common Crawl	Optimal artificial, Automatic deep features	Deep Learning	48	99.65
Dam Minh et. al [48] 2024	Malicious URLs	Extensions in Web browser	Deep Learning	-	98.4
Proposed Article	Custom	Heatmap+RFFIS	Keras Sequential Deep Learning	64	99.28

Also, as observed in the results from the tables 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, and 18 the following observations can be made: 1. Irrespective of the mode, optimised data performs better in evaluating performance metrics. The time complexity of the classifiers are also shown for each models and frameworks in the respective results tables. 2. Incremental mode has the added advantage of processing big data with less storage and computational resources, giving better results and thus being useful in wearable or resource-constrained devices. Tables 19 and 20 depict the state-of-the-art phishing detection systems by various researchers in terms of both accuracy and computational efficiency using phishing datasets, various feature extraction and selection techniques, and different models and classifiers applied.

V. CONCLUSION AND FUTURE WORKS

From the research conducted, the following are the contributions and novelty obtained: First, filtering out less relevant features or adding more optimal features gives us almost the same or better accuracy compared to all the features taken into consideration. This saves the overhead of preprocessing irrelevant features. Hence our study details the hybrid approach on how to select the optimal features on the phishing dataset to optimize the classification. Second, the experiment was conducted in offline, batch, and incremental modes for three different datasets, mainly to check its performance and efficiency in different modes of execution. This would be beneficial for end users to choose their modes of processing the data based on the type on data such as static or real time. A detailed pros and cons of each modes as well as

the advantages and disadvantages of each model applied are explained. Third, to avoid any kind of potential bias that could be prevalent in public datasets, a custom dataset was prepared and used in the research work. Fourth, the batch sizes in batch mode were varied to analyse the performance of the model on the phishing dataset. Fifth, ensemble feature extraction techniques were applied to fine-tune the feature selection process. The key findings of the study can be summarised as static and batch modes work better when the data set is small or if the data is not very volatile. The disadvantage of batch and offline modes is the cost burden on the processing units when the data size increases. The incremental mode is a suitable solution for computing real-time critical applications because of its computing and storage advantages. As seen in the results sections, the accuracy is not much compromised in the incremental learning, even though the attributes can be processed only once. One noteworthy downside in the case of incremental mode is that the features that need to be extracted for the study at one point in time may not be worthy of consideration the second time we try to compute the model or may not be available for a repeat study. But there are hopeful efforts and progress in incremental frameworks, such as River, to optimise the computations. One limitation of the River framework is that it works only for numerical data and not multimedia data such as images, audios, or videos. Hence, processing multimedia data in an incremental setting using the River framework is still a work in progress and gives way for future work. Also, we have shown how having different types of data, like in our case, the phishing dataset, which has many attributes, and the SMS spam ham dataset, which has sparse attributes, can affect the

classifier models. As a future work, we can implement the model using deep learning algorithms to optimise the result because deep learning does automatic feature extraction and has a huge scope for improvement. The results and methods applied in this study can be implemented in similar problems where data is numerical and streaming, like in the cases of financial banking transaction to identify if the transactions is genuine or a fraudulent one, or in identifying spam, phishing, or genuine emails.

REFERENCES

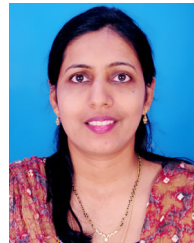
- [1] R. Kiruthiga and D. Akila, "Phishing websites detection using machine learning," *Int. J. Recent Technol. Eng.*, vol. 8, no. 2, pp. 111–114, Sep. 2019.
- [2] W. Ali, "Phishing website detection based on supervised machine learning with wrapper features selection," *Int. J. Adv. Comput. Sci. Appl.*, vol. 8, no. 9, pp. 72–78, 2017.
- [3] O. Abayomi-Alli, S. Misra, and A. Abayomi-Alli, "A deep learning method for automatic SMS spam classification: Performance of learning algorithms on indigenous dataset," *Concurrency Comput., Pract. Exper.*, vol. 34, no. 17, Aug. 2022, Art. no. e6989.
- [4] L. Tang and Q. H. Mahmoud, "A survey of machine learning-based solutions for phishing website detection," *Mach. Learn. Knowl. Extraction*, vol. 3, no. 3, pp. 672–694, Aug. 2021.
- [5] M. U. Togbe, M. Barry, A. Boly, Y. Chabchoub, R. Chiky, J. Montiel, and V. T. Tran, "Anomaly detection for data streams based on isolation forest using scikit-multiflow," in *Proc. Int. Conf. Comput. Sci. Appl.*, Cagliari, Italy, Cham, Switzerland: Springer, Jul. 2020, pp. 15–30.
- [6] P. Yang, G. Zhao, and P. Zeng, "Phishing website detection based on multidimensional features driven by deep learning," *IEEE Access*, vol. 7, pp. 15196–15209, 2019.
- [7] R. S. Rao, T. Vaishnavi, and A. R. Pais, "CatchPhish: Detection of phishing websites by inspecting URLs," *J. Ambient Intell. Humanized Comput.*, vol. 11, no. 2, pp. 813–825, Feb. 2020.
- [8] A. Aljofey, Q. Jiang, Q. Qu, M. Huang, and J.-P. Niyigana, "An effective phishing detection model based on character level convolutional neural network from URL," *Electronics*, vol. 9, no. 9, p. 1514, Sep. 2020.
- [9] J. V. Jawade and S. N. Ghosh, "Phishing website detection using FastAi library," in *Proc. Int. Conf. Commun. Inf. Comput. Technol. (ICCICT)*, Jun. 2021, pp. 1–5.
- [10] T. Bilot, G. Geis, and B. Hammi, "PhishGNN: A phishing website detection framework using graph neural networks," in *Proc. 19th Int. Conf. Secur. Cryptogr.*, 2022, pp. 428–435.
- [11] Y. A. Alsariera, V. E. Adeyemo, A. O. Balogun, and A. K. Alazzawi, "AI meta-learners and extra-trees algorithm for the detection of phishing websites," *IEEE Access*, vol. 8, pp. 142532–142542, 2020.
- [12] H. Musa, M. S. Adamu, A. Y. Gital, U. Ali, A. M. Kwami, F. U. Zambuk, and A. A. Aminu, "GAX and GAS modes: A new hybrid deep learning method for phishing URL detection using GRU, attention mechanisms, SVM and XGBOOST algorithms," *ATBU J. Sci., Technol. Educ.*, vol. 10, no. 3, pp. 426–436, May 9, 2023.
- [13] M. Alshehri, A. Abugabah, A. Algarni, and S. Almotairi, "Character-level word encoding deep learning model for combating cyber threats in phishing URL detection," *Comput. Electr. Eng.*, vol. 100, May 2022, Art. no. 107868.
- [14] A. Odeh and I. Keshta, "PhiBoost—A novel phishing detection model using adaptive boosting approach," *Jordanian J. Comput. Inf. Technol.*, vol. 7, no. 1, p. 64, 2021.
- [15] A. Safi and S. Singh, "A systematic literature review on phishing website detection techniques," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 35, no. 2, pp. 590–611, Feb. 2023.
- [16] C. C. L. Tan, K. L. Chiew, K. S. C. Yong, Y. Sebastian, J. C. M. Than, and W. K. Tiong, "Hybrid phishing detection using joint visual and textual identity," *Expert Syst. Appl.*, vol. 220, Jun. 2023, Art. no. 119723.
- [17] M. A. Adebawale, K. T. Lwin, and M. A. Hossain, "Intelligent phishing detection scheme using deep learning algorithms," *J. Enterprise Inf. Manage.*, vol. 36, no. 3, pp. 747–766, Apr. 2023.
- [18] W. Li, S. Manickam, S. U. A. Laghari, and Y.-W. Chong, "Uncovering the cloak: A systematic review of techniques used to conceal phishing websites," *IEEE Access*, vol. 11, pp. 71925–71939, 2023.
- [19] H. Baaqeel and R. Zagrouba, "Hybrid SMS spam filtering system using machine learning techniques," in *Proc. 21st Int. Arab Conf. Inf. Technol. (ACIT)*, Nov. 2020, pp. 1–8.
- [20] G. Ubale and S. Gaikwad, "SMS spam detection using TFIDF and voting classifier," in *Proc. Int. Mobile Embedded Technol. Conf. (MECON)*, Mar. 2022, pp. 363–366.
- [21] P. Joseph and S. Y. Yerima, "A comparative study of word embedding techniques for SMS spam detection," in *Proc. 14th Int. Conf. Comput. Intell. Commun. Netw. (CICN)*, Dec. 2022, pp. 149–155.
- [22] E. Shamsinejad, T. Baniroostam, M. M. Pedram, and A. M. Rahmani, "Presenting a model of data anonymization in big data in the context of in-memory processing framework," *J. Elect. Comput. Eng. Innov.*, vol. 12, no. 1, pp. 79–98, Jan. 2024.
- [23] N. Cummins, B. Killen, S. B. Ramezani, S. Rahimi, M. Seale, and S. Mittal, "A comparative study of continual, lifelong, and online supervised learning libraries," in *Proc. Int. FLAIRS Conf.*, vol. 36, May 2023, pp. 1–7.
- [24] T. Almeida and J. Hidalgo, *SMS Spam Collection*. Irvine, CA, USA: UCI Machine Learning Repository, 2012, doi: [10.24432/C5CC84](https://doi.org/10.24432/C5CC84).
- [25] F. Rustam, N. Saher, A. Mehmood, E. Lee, S. Washington, and I. Ashraf, "Detecting ham and spam emails using feature union and supervised machine learning models," *Multimedia Tools Appl.*, vol. 82, no. 17, pp. 26545–26561, Jul. 2023.
- [26] L. Melgar-García, D. Gutiérrez-Avilés, C. Rubio-Escudero, and A. Troncoso, "Identifying novelties and anomalies for incremental learning in streaming time series forecasting," *Eng. Appl. Artif. Intell.*, vol. 123, Aug. 2023, Art. no. 106326.
- [27] Q. Li, C. Qiu, M. Kloft, P. Smyth, M. Rudolph, and S. Mandt, "Zero-shot anomaly detection via batch normalization," in *Proc. Adv. Neural Inf. Process. Syst.*, Feb. 2024, pp. 1–37.
- [28] X. He, Y. Li, J. Tan, B. Wu, and F. Li, "OneShotSTL: One-shot seasonal-trend decomposition for online time series anomaly detection and forecasting," 2023, *arXiv:2304.01506*.
- [29] A. A. Orunsolu, S. O. Kareem, S. I. Salawu, K. S. Famuyiwa, and O. J. Elugbadebo, "An incremental component-based system for mitigating phishing attacks," *Acta Inf. Malaysia*, vol. 5, no. 2, pp. 53–57, Sep. 2021.
- [30] S. Giri, S. Das, S. B. Das, and S. Banerjee, "SMS spam classificationsimple deep learning models with higher accuracy using BUNOW and GloVe word embedding," *J. Appl. Sci. Eng.*, vol. 26, no. 10, pp. 1501–1511, 2023.
- [31] A. Prasad and S. Chandra, "PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning," *Comput. Secur.*, vol. 136, Jan. 2024, Art. no. 103545.
- [32] S. Jamal and H. Wimmer, "An improved transformer-based model for detecting phishing, spam, and ham: A large language model approach," 2023, *arXiv:2311.04913*.
- [33] S. Asiri, Y. Xiao, S. Alzahrani, S. Li, and T. Li, "A survey of intelligent detection designs of HTML URL phishing attacks," *IEEE Access*, vol. 11, pp. 6421–6443, 2023.
- [34] R. Alazaidah, A. Al-Shaikh, M. R. Al-Mousa, H. Khafajah, G. Samara, M. Alzyoud, N. Al-Shanableh, and S. Almatarnah, "Website phishing detection using machine learning techniques," *J. Statist. Appl. Probab.*, vol. 13, no. 1, pp. 119–129, 2024.
- [35] A. K. Jain and B. B. Gupta, "PHISH-SAFE: URL features-based phishing detection system using machine learning," in *Cyber Security*. Singapore: Springer, 2015, pp. 467–474.
- [36] S. R. A. Samad, S. Balasubramanian, A. S. Al-Kaabi, B. Sharma, S. Chowdhury, A. Mehbodniya, J. L. Webber, and A. Bostani, "Analysis of the performance impact of fine-tuned machine learning model for phishing URL detection," *Electronics*, vol. 12, no. 7, p. 1642, Mar. 2023.
- [37] Z. Alshingiti, R. Alaqel, J. Al-Muhtadi, Q. E. U. Haq, K. Saleem, and M. H. Faheem, "A deep learning-based phishing detection system using CNN, LSTM, and LSTM-CNN," *Electronics*, vol. 12, no. 1, p. 232, Jan. 2023.
- [38] A. Karim, M. Shahroz, K. Mustofa, S. B. Belhaouari, and S. R. K. Joga, "Phishing detection system through hybrid machine learning based on URL," *IEEE Access*, vol. 11, pp. 36805–36822, 2023.
- [39] M. Babagoli, M. P. Aghababa, and V. Solouk, "Heuristic nonlinear regression strategy for detecting phishing websites," *Soft Comput.*, vol. 23, no. 12, pp. 4315–4327, Jun. 2019.
- [40] L. Al-Shalabi and Y. H. Jazyah, "Detection using hybrid algorithm based on clustering and machine learning," *Int. J. Comput. Digit. Syst.*, vol. 15, no. 1, pp. 1–3, Jan. 2024.

- [41] R. S. Rao, A. R. Pais, and P. Anand, "A heuristic technique to detect phishing websites using TWSVM classifier," *Neural Comput. Appl.*, vol. 33, no. 11, pp. 5733–5752, Jun. 2021.
- [42] A. Subasi, E. Molah, F. Almkallawi, and T. J. Chaudhery, "Intelligent phishing website detection using random forest classifier," in *Proc. Int. Conf. Electr. Comput. Technol. Appl. (ICECTA)*, Nov. 2017, pp. 1–5.
- [43] M. T. Suleman and S. M. Awan, "Optimization of URL-based phishing websites detection through genetic algorithms," *Autom. Control Comput. Sci.*, vol. 53, no. 4, pp. 333–341, Jul. 2019.
- [44] A. Frank, "UCI machine learning repository," Irvine, CA, USA: Univ. of California, School of Information and Computer Science, 2010. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [45] R. J. van Geest, G. Cascavilla, J. Hulstijn, and N. Zannone, "The applicability of a hybrid framework for automated phishing detection," *Comput. Secur.*, vol. 139, Apr. 2024, Art. no. 103736.
- [46] E. Kritika, "A comprehensive literature review on phishing URL detection using deep learning techniques," *J. Cyber Secur. Technol.*, vol. 15, pp. 1–29, Jul. 2024.
- [47] E. Zhu, K. Cheng, Z. Zhang, and H. Wang, "PDHF: Effective phishing detection model combining optimal artificial and automatic deep features," *Comput. Secur.*, vol. 136, Jan. 2024, Art. no. 103561.
- [48] D. M. Linh, H. D. Hung, H. M. Chau, Q. S. Vu, and T.-N. Tran, "Real-time phishing detection using deep learning methods by extensions," *Int. J. Electr. Comput. Eng. (IJECE)*, vol. 14, no. 3, p. 3021, Jun. 2024.
- [49] P. Pandey and N. Mishra, "Phish-sight: A new approach for phishing detection using dominant colors on web pages and machine learning," *Int. J. Inf. Secur.*, vol. 22, no. 4, pp. 881–891, Aug. 2023.
- [50] S. Ariyadasa, S. Fernando, and S. Fernando, "SmartiPhish: A reinforcement learning-based intelligent anti-phishing solution to detect spoofed website attacks," *Int. J. Inf. Secur.*, vol. 23, no. 2, pp. 1055–1076, Apr. 2024.
- [51] P. Bountakas and C. Xenakis, "HELPHED: Hybrid ensemble learning PHishing email detection," *J. Netw. Comput. Appl.*, vol. 210, Jan. 2023, Art. no. 103545.
- [52] Y. Mourtaji, M. Bouhorma, D. Alghazzawi, G. Aldabbagh, and A. Alghamdi, "hybrid rule-based solution for phishing URL detection using convolutional neural network," *Wireless Commun. Mobile Comput.*, vol. 2021, no. 1, Jan. 2021, Art. no. 8241104.
- [53] *Phishing Website Dataset*. Accessed: May 10, 2024. [Online]. Available: <https://www.kaggle.com/datasets/akashkr/phishing-website-dataset>
- [54] *Phishing*. Accessed: May 10, 2024. [Online]. Available: <https://www.kaggle.com/datasets/divyajenniferdsouza1/phishing>
- [55] *SMS Spam Collection Dataset*. Accessed: May 10, 2024. [Online]. Available: <https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>
- [56] H. Qin, X. Zhang, R. Gong, Y. Ding, Y. Xu, and X. Liu, "Distribution-sensitive information retention for accurate binary neural network," *Int. J. Comput. Vis.*, vol. 131, no. 1, pp. 26–47, Jan. 2023.
- [57] H. Qin, Y. Ding, X. Zhang, J. Wang, X. Liu, and J. Lu, "Diverse sample generation: Pushing the limit of generative data-free quantization," *IEEE Trans. Pattern Anal. Mach. Intell.*, pp. 1–18, 2023.
- [58] H. Qin, X. Ma, X. Zheng, X. Li, Y. Zhang, S. Liu, J. Luo, X. Liu, and M. Magno, "Accurate LoRA-finetuning quantization of LLMs via information retention," 2024, *arXiv:2402.05445*.
- [59] H. Qin, Y. Zhang, Y. Ding, X. Liu, M. Danelljan, and F. Yu, "QuantSR: Accurate low-bit quantization for efficient image super-resolution," in *Proc. Adv. Neural Inf. Process. Syst.*, Feb. 2024, pp. 1–11.
- [60] S. M. Ahmadi, S. Balahang, and S. Abolfathi, "Predicting the hydraulic response of critical transport infrastructures during extreme flood events," *Eng. Appl. Artif. Intell.*, vol. 133, Jul. 2024, Art. no. 108573.
- [61] A. Yeganeh-Bakhtiary, H. EyvazOghli, N. Shabakhty, and S. Abolfathi, "Machine learning prediction of wave characteristics: Comparison between semi-empirical approaches and DT model," *Ocean Eng.*, vol. 286, Oct. 2023, Art. no. 115583.
- [62] J. Moedjahedy, A. Setyanto, F. K. Alarfaj, and M. Alreshoodi, "CCrFS: Combine correlation features selection for detecting phishing websites using machine learning," *Future Internet*, vol. 14, no. 8, p. 229, Jul. 2022.
- [63] R. G. Jimoh, A. M. Oyelakin, O. C. Abikoye, M. B. Akanbi, A. O. Akanni, M. A. Jibrin, and T. S. Ogundele, "Efficient ensemble-based phishing website classification models using feature importance attribute selection and hyper parameter tuning approaches," *J. Inf. Technol. Comput.*, vol. 4, no. 2, pp. 1–10, Dec. 2023.



DIVYA JENNIFER DSOUZA received the B.E. and M.Tech. degrees in computer science engineering from VTU, Belagavi, India, in 2009 and 2015, respectively. She is currently pursuing the Ph.D. degree with the NMAM Institute of Technology, NITTE (Deemed to be University), Nitte, India.

From 2010 to 2013, she was a Systems Engineer with Infosys Technology. From 2013 to 2022, she was an Assistant Professor with the NMAM Institute of Technology. She is the author of 16 research articles. Her research interests include cyber security, machine learning, and deep learning.



ANISHA P. RODRIGUES was born in Mangaluru, India, in 1985. She received the B.E. degree in electrical and electronics engineering, the M.Tech. degree in computer science and engineering, and the Ph.D. degree in big data analytics from VTU, Belagavi, India, in 2006, 2012, and 2020, respectively. Since 2012, she has been an Associate Professor with the Department of Computer Science and Engineering, NMAM Institute of Technology, NITTE (Deemed to be University),

India. She is the author of three book chapters and 30 articles. Her research interests include natural language processing, machine learning, deep learning, data science, and big data analytics.



ROSHAN FERNANDES was born in Mangaluru, India, in 1979. He received the B.E. degree in computer science and engineering from Mangalore University, in 2001, and the M.Tech. degree in computer engineering and the Ph.D. degree in mobile web services from VTU, Belagavi, India, in 2007 and 2019, respectively. Currently, he is a Professor and the Head with the Department of Cyber Security, NMAM Institute of Technology, NITTE (Deemed to be University), India. He has

a teaching experience of 20 years in this institution. He is the author of four book chapters and 20 articles. His research interests include mobile web services, cyber security, natural language processing, machine learning, and deep learning.

...